

Prepify – A Community Driven Platform for Handwritten Academic Notes

By
Shorno Kamal Roy
ID: 222-15-6141

FINAL YEAR DESIGN PROJECT REPORT

**This Report Presented in Partial Fulfillment of the
Requirements for the Degree of Bachelor of Science in
Computer Science and Engineering**

Supervised by

**Md. Sazzadur
Ahamed,
Assistant
Professor**
Department of Computer Science and
Engineering Daffodil International
University

Co-Supervised by

**Md. Ashraful
Islam Talukder,
Lecturer**
Department of Computer Science and
Engineering Daffodil International
University



**DAFFODIL INTERNATIONAL
UNIVERSITY**
Dhaka, Bangladesh

April 30, 2026

APPROVAL

This Project titled “Prepify – A Community Driven Platform for Handwritten Academic Notes,” submitted by Shorno Kamal Roy to the Department of Computer Science and Engineering, Daffodil International University, has been accepted as satisfactory for the partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Engineering and approved as to its style and contents. The presentation has been held on 26-01-2024.

BOARD OF EXAMINERS

Name

Board Chairman

Designation, Department of
CSE, FSITDaffodil
International University

Name

Internal Examiner 1

Designation, Department of
CSE, FSITDaffodil
International University

Name

Internal Examiner 2

Designation, Department of
CSE, FSITDaffodil
International University

Name

External Examiner

Designation, Department of
CSE, FSITDaffodil
International University

DECLARATION

I hereby declare that this project has been done by me under the supervision of Md. Sazzadur Ahamed, Assistant Professor, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere for the award of any degree or diploma.

Supervised by:

Md. Sazzadur Ahamed

Assistant Professor

Department of Computer Science and
Engineering Daffodil International
University

Co-Supervised by:

Md. Ashraful Islam Talukder

Lecturer

Department of Computer Science and
Engineering Daffodil International
University

Submitted by:

Shorno Kamal Roy

Student ID: 222-15-6141

Department of Computer Science and
Engineering Daffodil International
University

Student ID:

Department of Computer Science and
Engineering Daffodil University

ACKNOWLEDGEMENTS

This work would not have been possible without the support and contributions of many individuals over the past two semesters. I am deeply grateful to everyone who has assisted us in one way or another.

First, I express my heartfelt thanks and gratefulness to the almighty for His divine blessing making it possible for me to complete the Final Year Design Project(FYDP) successfully.

I am grateful and wish my profound indebtedness to Md. Sazzadur Ahamed, Assistant Professor, Department of Computer Science and Engineering, Daffodil International University, Dhaka, Bangladesh. Deep knowledge and keen interest of my supervisor in the field of web application development and AI-powered educational technology to carry out this project. His endless patience, scholarly guidance, continual encouragement, constant and energetic supervision, constructive criticism, valuable advice, reading many inferior drafts, and correcting them at all stages have made it possible to complete this project.

I would like to express my heartfelt gratitude to the Head of the Department of Computer Science and Engineering, for his kind help in finishing my project and also to other faculty members and the staff of the Department of Computer Science and Engineering, Daffodil International University.

I would like to thank my course-mates at Daffodil International University, who took part in this discussion while completing the coursework.

Finally, I must acknowledge with due respect the constant support and patience of my parents.

ABSTRACT

Prepify is a web-based platform built for university students who want to share, organize, and discover handwritten academic notes. In Bangladesh and similar regions, students commonly exchange study materials through informal messaging groups on WhatsApp, Telegram, or Facebook. These notes get buried under newer messages, are hard to search, and vary wildly in quality. Prepify addresses this by providing a structured space where handwritten solutions can be uploaded as photographs, categorized by department and course, and ranked through community voting. The platform is built on Next.js 16.1 and React 19.2 for the front end, PostgreSQL with Drizzle ORM for data storage, and Cloudinary for image hosting. Better Auth with Google OAuth is used for authentication. A points-based leaderboard encourages quality contributions, and a moderation workflow with admin and moderator dashboards keeps content standards high. The platform also uses OCR technology to pull text from handwritten images and large language model APIs to turn rough notes into well-organized study material. Development followed an Agile methodology with iterative prototyping over two semesters. Phase 1 delivered the core platform with authentication, note upload, voting, commenting, follow notifications, and moderation. Phase-2 focuses on the AI pipeline for content extraction and enhancement. Testing confirms reliable handling of multi-image uploads, real-time vote updates, and content moderation workflows.

Table of Contents

Approval	i
Declaration	ii
Acknowledgements	iii
Abstract	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Introduction.....	1
1.2 Motivation	1
1.3 Objectives	1
1.4 Methodology	1
1.5 Project Outcome.....	1
1.6 Organization of the Report	1
2 Background	2
2.1 Introduction.....	2
2.2 Literature Review	2
2.2.1 Similar Applications	3
2.3 Gap Analysis	3
2.4 Summary	3
3 Research Methodology	4
3.1 Methodology/Requirement Analysis & Design Specification.....	4
3.1.1 Overview	4
3.1.2 Proposed Methodology/ System Design	4
3.1.3 Functional and Nonfunctional Requirements	5
3.1.4 Data Flow Diagram	5
3.1.5 UI Design	5
3.2 Detailed Methodology and Design	5
3.3 Project Plan	5
3.4 Task Allocation.....	5
3.5 Summary	5

4	Implementation and Results	6
4.1	Environment Setup	6
4.2	Testing and Evaluation/Performance/ Comparative Analysis	6
4.3	Results and Discussion	6
4.4	Summary	6
5	Engineering Standards and Design Challenges	7
5.1	Compliance with the Standards.....	7
5.1.1	Software Standards.....	7
5.1.2	Hardware Standards	7
5.1.3	Communication Standards.....	7
5.2	Impact on Society, Environment and Sustainability	7
5.2.1	Impact on Life.....	7
5.2.2	Impact on Society & Environment.....	7
5.2.3	Ethical Aspects	7
5.2.4	Sustainability Plan.....	7
5.3	Project Management and Financial Analysis.....	7
5.4	Complex Engineering Problem.....	8
5.4.1	Complex Problem Solving.....	8
5.4.2	Engineering Activities.....	8
5.5	Summary	8
6	Conclusion	10
6.1	Summary	10
6.2	Limitation	10
6.3	Future Work	10
	References	11

List of Figures

3.1	System Architecture of Prepify	13
3.2	Entity Relationship Diagram.....	13
3.3	Use Case Diagram.....	14
3.4	Data Flow Diagram.....	17
3.5	Landing page	19
3.6	Note details page.....	19
3.7	Note Upload page	20
3.8	Leaderboard page.....	20
3.9	Note Upload page	20
3.8	Agile Methodology Cycle + Phase Timeline.....	21
4.1	Authentication Flow	27
4.2	Like and Leaderboard Transaction Flow.....	29
4.3	Note Moderation State Machine.....	32
4.4	AI Explanation Pipeline	34

List of Tables

2.1	Summary of Literature Reviewed.....	6
2.2	Comparative Analysis of Similar Platforms.....	8
5.1	Estimated Cost and Financial Analysis.....	44
5.2	Mapping with Complex Engineering Problem..	45
5.3	Mapping with knowledge Profile.....	46
5.4	Mapping with Complex Engineering Activities.....	47

Chapter 1

Introduction

This chapter introduces the background and motivation behind Prepify, defines the project objectives, outlines the development methodology, and describes the expected outcomes.

1.1 Introduction

Exam preparation in most Bangladeshi universities still revolves around handwritten notes. Students take notes during lectures, photograph them with their phones, and then circulate the photos in informal groups on WhatsApp, Telegram, and Facebook Messenger. This pattern works in small batches but breaks down very quickly. A note that helped one student last semester is almost impossible to find six months later because it has been pushed deep below thousands of unrelated messages. There is no search, no tagging, no way of knowing whether a given page is from the right course, and no quality signal to tell a careful solution apart from a hurriedly copied one. The same handwritten solution often gets shared and re-shared across batches, with each forward losing context until the source is forgotten. The cost of this fragmentation falls mostly on weaker or quieter students who do not happen to be in the right group at the right time. Notes that exist somewhere on a senior's phone never reach them. The students who do collect these notes often hoard them, partly because the act of finding good notes feels like a personal achievement worth protecting. The result is that good academic content stays trapped in private galleries and dies there once the device is replaced.

Prepify approaches this as a structuring and discovery problem rather than as a content shortage problem. The notes already exist. What does not exist is a place where they can be uploaded once, organized by university and course, ranked by the community, and then revisited at any point in the future. The platform gives every student a public profile, a feed of recently approved notes, a search and filter interface that goes through the academic hierarchy of faculty, department, and course, and a leaderboard that tracks who has contributed the most useful material.

What sets Prepify apart from a generic note repository is its AI explanation pipeline. When a note is approved on the platform, the uploader can ask the system to analyze the photographed pages directly with a multimodal large language model. The model returns a clean summary of what the solution covers, a numbered step by step breakdown of the approach used in the handwriting, the key formulas or theorems that appear in the notes, any observations about possible errors or alternative methods, and a set of topic tags for discoverability. A student looking at a slightly messy handwritten solution can therefore read a structured digital explanation alongside the original page, which is something neither informal sharing nor commercial study platforms currently offer for free in this region.

1.2 Motivation

The motivation for Prepify came out of personal frustration during exam weeks. Every semester, the same scene repeats itself. Group chats fill up with frantic requests for handwritten solutions. Someone replies with a photo, the photo is too blurry, someone else asks for a clearer version, the conversation moves on, and a week later nobody remembers which chat had which file. The students who do find the right material end up sending it to their close circles in private chats, which makes the situation worse for everyone outside that circle.

There were three observations that turned this frustration into a project idea. First, smartphone cameras are now good enough that a quick photo of a handwritten page can be read clearly even after compression. The technology bottleneck is no longer image quality. Second, large language models have moved from text only systems to multimodal models that can read images directly, including handwritten pages, in both English and Bangla. This means a model can take a photo of someone's notebook and return a structured digital explanation without a separate OCR step in the middle. Third, students already trust the act of sharing notes with each other. They do not need to be convinced that handwritten notes are valuable. They only need a better surface than a chat thread to share them on.

There is also a fairness angle that pushed the project forward. The students who are best connected end up with the best notes. The students who are quieter, who joined the university through a transfer batch, or who simply do not enjoy networking, often end up with no notes at all. A platform that makes good notes findable through the academic structure rather than through the social structure should reduce that gap.

Existing platforms do not address this gap well. Docsity and Studocu host typed lecture content behind partial paywalls. Thinkswap uses a credit based exchange. Chegg and Course Hero are subscription services priced for Western markets. The closest local example, Ocenot, was a paid marketplace and is no longer available. None of these platforms specifically focus on photographed handwritten content from Bangladeshi universities, and none of them combine free access with AI enhancement of handwriting.

That combination is what makes the problem worth solving here.

1.3 Objectives

The objectives of this project are as follows:

- i. Develop a collaborative web platform for uploading, organizing, and sharing handwritten academic notes.
- ii. Implement community engagement features such as voting, ranking, and a contributor leaderboard.
- iii. Prepare an AI integration framework for OCR, summarization, and quiz generation from handwritten notes.

1.4 Methodology

The project follows an Agile development methodology with iterative sprints. The full work has been split into two phases mapped to the two FYDP semesters. Phase 1 covered the social platform layer, and Phase 2 added the AI driven features along with the bookmark and gamification subsystems. Each sprint delivered a usable increment of the product and was followed by an internal review against the requirements.

The technology stack was chosen to keep the operational cost low while still allowing modern UI and AI integration. Next.js 16.1 with React 19.2 provides server side rendering through the App Router for fast initial loads, with client components handling interactive parts such as voting, commenting, and bookmarking. Server logic is implemented through Next.js Server Actions, which removes the need for a separate API server and reduces the overall surface area of the system. PostgreSQL is hosted on Neon's serverless platform and accessed through Drizzle ORM, which gives close to SQL syntax while still being type safe. User images are uploaded to Cloudinary using signed presets and served through its CDN with on the fly format and quality optimizations. Better Auth handles authentication with Google OAuth as the only sign in method, and Brevo handles transactional email through its REST API. State management uses Zustand for short lived client state such as optimistic vote updates, and TanStack Query for server cache synchronization. The AI explanation pipeline is built on the Vercel AI SDK with Google's gemini-2.5-flash model, configured to return strictly typed structured output validated by a Zod schema.

The development process itself was supported by Git for version control and a single GitHub repository. Linting with ESLint and TypeScript strict mode was enabled from day one. Drizzle Kit handled migration generation, and Drizzle Studio was used to inspect the database during development.

1.5 Project Outcome

The completed platform allows any student with a Google account to sign up, complete a short onboarding that captures their faculty, department, and batch, and then immediately start uploading or browsing handwritten notes. Each upload supports several images at once, an optional set of reference URLs, a description, and a course assignment. Once a note is approved by a moderator, it becomes visible on the public feed, can be liked, commented on, bookmarked into a private or public collection, and can have an AI explanation generated for it.

The leaderboard recalculates contributor points whenever an approved note receives a like, with the points stored in a transaction log so that any future audit can trace each point back to its source event. The badges system awards milestone badges across upload, streak, engagement, and social categories. Streaks are tracked daily using a date string field that is updated atomically by the streak action.

For administrators, the platform exposes a dashboard with platform wide statistics, a user management page with role assignment, a moderator application review page, and a moderation queue. Moderators see a focused queue of pending notes with the original images, the metadata, and approve or reject controls. Rejected notes carry a reason that is then displayed to the uploader on their My Notes page.

The AI explanation feature returns a structured JSON object from Gemini 2.5 Flash containing a one or two sentence summary, an ordered list of solution steps, the key concepts identified in the handwriting, observations such as possible errors or alternate approaches, and topic tags for discovery. Each note can be regenerated up to three times to discourage abuse while allowing for genuine retries on failed runs.

The interface adapts cleanly from a 375 px mobile screen up to a 1920 px desktop, supports system level light and dark themes, and has been measured at scores above 85 on Lighthouse for performance and above 90 for accessibility on the production

deployment.

1.6 Organization of the Report

The remainder of the report has been arranged into five further chapters. Chapter 2 reviews the academic literature and the closest commercial platforms, identifies the specific gaps in the current landscape, and explains how Prepify's design responds to those gaps. Chapter 3 describes the system design, the functional and non functional requirements, the data flow, the user interface, the alternatives that were considered for each major component, the project plan, and the task allocation timeline. Chapter 4 presents the development environment, walks through the implementation of every major module including the AI explanation pipeline, summarizes the testing approach used, and discusses the results obtained on the production deployment. Chapter 5 covers compliance with software, hardware, and communication standards, the social, environmental, and ethical impact of the platform, the financial analysis, and the mapping to complex engineering problem categories. Chapter 6 closes the report with a summary of what has been delivered, an honest discussion of the current limitations, and a list of directions for future work.

Chapter 2

Background

This chapter places the project in the wider context of educational technology. It first reviews the academic literature on community driven learning environments, OCR and multimodal models, and gamification of student platforms. It then examines the closest commercial competitors. Finally it summarizes the gaps that emerged from the review and shows how Prepify aims to fill them.

2.1 Introduction

Educational technology has changed rapidly over the last decade. Online learning platforms, digital note repositories, and AI assisted tutoring tools are now common in universities around the world. Most of this work has been built around typed content, structured documents, video lectures, and standardized question banks. Handwritten notes, despite still being the primary study material in many countries, have been left out of almost every major platform.

In countries like Bangladesh, the gap is sharp. Students rely on physical notebooks for class notes and on photographs of those notebooks for sharing. The infrastructure they use for sharing, however, is the same set of consumer messaging apps used for everything else. There is no academic structure to the conversation, no quality control, and no permanent archive. As a result, study material that is genuinely valuable has a very short useful life before it gets buried.

The technology required to fix this has only recently become practical. Modern smartphones produce images good enough for distant readers, multimodal large language models can read handwritten content directly without a separate OCR step, and serverless infrastructure makes it possible to run this kind of platform at almost no cost while a user base is small. These three shifts together create the opportunity that Prepify takes advantage of [1].

2.2 Literature Review

A number of research works have studied the broader ideas behind this project, ranging from how learners benefit from personalized environments, to how OCR and multimodal models perform on handwritten material, to how community ranking can drive content quality.

Chatti et al. [1] proposed the Personal Learning Environment (PLE) framework. They argued that learners benefit when they are given personalized spaces in which they can collect, organize, and share educational resources, instead of being forced to rely on a single rigid institutional system. Their reasoning supports the design choice in Prepify, where the user controls what they upload, how they tag it, and which collections they keep private. Attwell [2] looked at the future of e learning through the lens of personal learning environments and concluded that students perform better when they have control over their resources and can share them inside trusted communities. This aligns with the community moderation and voting approach used in Prepify.

Getenet et al. [3] studied student attitudes toward digital technology in online learning

and found that students engage more strongly with platforms that feel close to their existing habits. Since most Bangladeshi students already share photographed handwritten notes informally, Prepify deliberately builds on that familiar behaviour rather than replacing it with something unfamiliar.

For the AI side, Google DeepMind [4] introduced the Gemini family of multimodal models, which can process text, images, audio, and video in a single conversation. The Gemini 2.5 Flash model used in this project belongs to that family and is capable of reading handwritten pages directly. Smith [5] gave an overview of the Tesseract OCR engine, which has historically been the most widely used open source OCR tool. Tesseract remains useful for printed text but tends to struggle on irregular handwriting, which is one of the reasons Prepify uses a multimodal model instead of an OCR plus LLM pipeline. Vaswani et al. [6] introduced the Transformer architecture in their seminal paper on self attention. This architecture underpins almost every modern large language model, including Gemini, and is the structural reason a single model can read images and produce structured text simultaneously.

On the platform side, Docsity [7] is a global note sharing site with rating and reputation features but with most premium content behind a paywall and no specific focus on handwriting. Thinkswap [8] uses a credit based exchange model that effectively forces students to upload before they can download. Studocu [9] hosts typed academic documents with a freemium tier and also blocks handwritten material behind premium walls. Course Hero and Chegg [10][11] operate as paid subscription services oriented toward Western markets. Ocenot [12] was a Bangladesh specific marketplace where students bought and sold notes; the platform was paid only and is no longer reachable. None of these platforms combine free access, focus on handwriting, community ranking, and AI assistance, which is the intersection that Prepify targets.

Vercel's documentation on Next.js [13] and the official Drizzle ORM documentation [14] informed the architectural decisions of the project, particularly the use of Server Actions and serverless databases. The Cloudinary developer documentation [15] guided the implementation of signed uploads and on the fly image transformations.

Table 2.1: Summary of Literature Reviewed.

Author (s)	Year	Title	Methodology	Key Findings
Chatti et al.[1]	2010	Toward a Personal Learning Environment Framework	Framework Analysis	Learners need personalized spaces to organize and share educational resources.
Attwell [2]	2007	Personal Learning Environments - the future of eLearning?	Literature Review	Students perform better with control over learning resources in communities.
Getenet et al. [3]	2024	Students digital technology attitude and online learning engagement	Survey-based	Students engage more with platforms relevant to existing study habits.

Google DeepMind [4]	2023	Gemini: A Family of Highly Capable Multimodal Models	Technical report	Multimodal models can directly process handwritten images and produce structured text
Smith [5]	2007	An overview of the Tesseract OCR engine	System description	Tesseract is robust on printed text but weaker on irregular handwriting
Vaswani et al. [6]	2017	Attention Is All You Need	Experimental	Introduced the Transformer architecture underpinning modern LLMs.
Docsity [7]	2025	Study Notes and University Resources	Commercial platform	Freemium typed note sharing with rating systems
Thinkswap [8]	2025	Study Notes and Learning Resources	Commercial platform	Credit based exchange limits free access
Studocu [9]	2025	Study Documents and Notes	Commercial platform	Freemium typed academic content
Ocenot [10]	2025	Buy and Sell Academic Notes	Commercial platform (offline)	Paid marketplace approach to local notes

2.2.1 Similar Applications (Mandatory for project, optional for research)

Several existing services aim at the same broad audience as Prepify, but each of them takes a different approach and serves a different gap.

Docsity is the most globally visible competitor. It hosts a very large library of student notes, lecture slides, and past papers across many universities, with a rating system for contributors and a reputation score that grows with usage. The platform also offers an AI study app that produces summaries and quizzes from documents. The two main limitations from a Bangladeshi student's perspective are that most useful AI features sit behind a premium subscription, and that the platform's category structure does not represent local universities, departments, or courses. A student looking for a specific Bangladeshi course like CSE 313 will not find it categorized as such on Docsity.

Thinkswap operates a credit based exchange. A student must upload their own material to earn credits, and those credits are then used to download other students' notes. This produces a reasonable amount of upload activity, but it also locks out students who genuinely need notes but do not have anything of their own to contribute yet. The platform also focuses on typed and scanned documents, with very little support for handwritten work.

Studocu has a freemium model where many notes are free to read but premium content and AI features require a paid subscription. It is heavily oriented toward Western universities and toward typed documents, similar to Docsity.

Course Hero and Chegg are paid subscription platforms. They have strong AI tutoring features and large libraries, but their pricing is structured for Western markets and is far above what most Bangladeshi students would pay for this kind of service.

Ocenot was a Bangladeshi marketplace where students could buy and sell handwritten and typed notes. It was the closest local match in terms of audience but operated as a transactional marketplace rather than a community platform, and the website is no longer accessible at the time of writing. It also did not offer any AI features.

Outside the note sharing space, a number of general purpose AI study assistants such as ChatGPT and Gemini have started to be used by students for explanations, but these are general chat interfaces rather than community platforms. They do not store, organize, or rank notes, and they treat each request as an isolated conversation.

The way these platforms position themselves makes the gap clearer. There is no platform that combines free access, focus on photographed handwritten content, community ranking with a contributor leaderboard, and AI explanation of handwriting through multimodal models. This intersection is what Prepify is built around.

2.3 Gap Analysis

The review of existing platforms reveals four key gaps that Prepify addresses. First, no platform offers completely free and open access to handwritten academic notes. Docsity operates on a freemium model where AI features and premium content require payment. Thinkswap uses a credit-based system where students must upload their own materials before downloading others. Ocenot functioned as a paid marketplace where notes were bought and sold. Prepify removes all access barriers entirely, allowing any student to upload, browse, and download handwritten notes without credits, subscriptions, or purchases. Second, no platform is designed specifically around handwritten academic content. Docsity and Thinkswap treat handwritten notes as just another file type alongside typed documents, lecture slides, and textbook solutions. Their search, organization, and AI features are optimized for typed or digital-first content. Prepify is built from the ground up for photographed handwritten solutions, with the entire upload workflow, OCR pipeline, and AI processing tailored for handwriting. Third, there is no platform that serves Bangladeshi university students with localized organization. Docsity and Thinkswap are global platforms where Bangladeshi university courses, departments, and exam patterns are not represented in their category structures. Ocenot was Bangladesh-focused but operated as a commercial marketplace rather than a community platform, and its website is no longer accessible. Prepify organizes content by Bangladeshi universities, departments, and specific courses, making it directly relevant to the local academic context. Fourth, no platform integrates AI processing of handwritten content directly into a free community workflow. While Docsity offers AI summarization and quiz generation through a separate app, these features are behind a paywall and designed for typed documents rather than handwritten notes. Thinkswap and Ocenot offer no AI features at all. Prepify integrates OCR text extraction, AI-powered summarization, and quiz generation directly into the note upload flow at no cost to the user, specifically targeting handwritten academic content.

Table 2.2: Comparative Analysis of Similar Platforms

Features	Docsity	Thinkswap	Ocenot	Prepify
Free note upload	Yes (Points)	Yes (Credits)	No (Paid)	Yes
Handwritten note focus	No	No	Yes	Yes
Course-based organization	Yes	Yes	Yes	Yes
Community voting/ranking	Yes	Yes (Ratings)	No	Yes
Contributor leaderboard	Yes	No	No	Yes
Content moderation	Yes	Yes	Manual	Yes
OCR for handwriting	Yes (App)	No	No	Yes
AI summarization	Yes (App)	No	No	Yes
Quiz generation from notes	Yes (App)	No	No	Yes
Follow/notification system	Yes	No	No	Yes
Dark mode support	No	No	No	Yes
Bangladesh-focused	No	No	Yes	Yes
Open/free access	Freemium	Freemium	No	Yes
Responsive mobile design	Yes	Yes	Partial	Yes

2.4 Summary

This chapter reviewed the academic literature on personal learning environments, OCR systems, multimodal language models, and community driven note sharing. It then surveyed six closely related commercial platforms and showed that none of them combine free access, focus on handwriting, community ranking, and AI assistance. The gap analysis identified five concrete gaps, each of which Prepify addresses by design rather than by patch. The next chapter describes the methodology and the system design used to build the platform around these gaps.

Chapter 3

Research Methodology

This chapter describes how the platform was designed, what requirements drove the design, how the data flows through the system, and how the work was scheduled across the two FYDP semesters. The chapter starts at the high level methodology, narrows down through the system architecture and the requirements, then ends on the project plan and timeline

3.1 Requirement Analysis & Design Specification

The work followed an Agile methodology with iterative prototyping. Each sprint lasted between one and two weeks, and each sprint ended with a working increment of the platform that could be opened in the browser, signed into, and used end to end. This style of development was a deliberate choice. The user expectations on a community platform are very sensitive to feel and to small interaction details, and these are not easy to specify upfront in a waterfall style document. By keeping the development cycle short, every UX decision could be tested directly against the running product instead of against a paper specification.

Requirements were gathered through three sources. The first was direct observation of student behaviour in the existing informal note sharing groups. This pointed to obvious pain points such as the difficulty of finding old notes, the duplication of similar requests, and the lack of any quality signal. The second source was a feature audit of the closest commercial platforms, which produced the comparison shown in the previous chapter. The third was the FYDP supervisor's review at each milestone, which often suggested concrete additions such as the moderator application flow and the badge system.

The system has been designed to be modular. The social platform layer (authentication, notes, voting, comments, follows, moderation) was developed first and treated as a stable surface. The AI explanation pipeline, the bookmarks subsystem, and the gamification layer (streaks and badges) were then added on top without restructuring the core. Each feature has its own database tables and its own server actions, and they communicate with the core through well defined relations rather than through tight coupling.

3.1.1 Overview

The platform is a multi user web application that follows a three tier pattern. The client tier is a Next.js application running React server and client components. The application tier is a set of Next.js Server Actions, which run on the server but are invoked directly from the client without a separate API endpoint definition. The data tier is a serverless PostgreSQL database hosted on Neon, with image assets stored on Cloudinary and cached at its edge network.

The user journey starts at the landing page, which works without any authentication. A visitor can see recently approved notes, browse departments, and read the platform statistics, but cannot upload, like, comment, or generate AI explanations until they sign in. Sign in itself is handled through a single Google OAuth button. After the first sign in, the user is taken through a short onboarding form that captures username, faculty, department, batch, and theme preference. Once onboarding is complete, the user lands on the main feed and can use any feature for which they are authorized.

Roles are kept simple. The default role is `user`. The `moderator` role is granted by an administrator either through the user management page or by approving a moderator application that the user has submitted through the dedicated form. The `admin` role is reserved for the platform owner and is set directly in the database. Each role unlocks additional pages that are otherwise hidden through layout level guards.

3.1.2 System Design

The architecture has been kept deliberately flat. There is one Next.js application that serves both the public website and the authenticated dashboards. There is no separate backend service. There is no GraphQL layer. The same code base contains the React UI, the Server Actions that perform reads and writes, the database schema declarations, and the AI integration. This keeps the cognitive load low and makes onboarding new contributors much easier in the future.

On the data side, the schema is normalized around a small number of core entities: users, faculties, departments, batches, courses, notes, files, resources, comments, likes, follows, notifications, leaderboard transactions, moderator applications, bookmarks, collections, streaks, badges, user badges, and AI explanations. Each of these is declared in its own Drizzle schema file under the db/schema directory and exported through a single barrel file. The relations between them are declared explicitly so that Drizzle's query builder can produce typed nested fetches without manual joins.

On the image side, all uploads go through Cloudinary. The browser obtains a signed upload preset from a server action, uploads directly to Cloudinary's API, and stores only the resulting URL in the database. This avoids the application server ever having to handle a large file body, which keeps latency low and the codebase simple. Cloudinary then serves the image through its CDN with on the fly format negotiation, so that a desktop browser receives a WebP version while an older mobile browser falls back to JPEG.

On the AI side, the platform calls Gemini 2.5 Flash through the Vercel AI SDK's generateObject function. A Zod schema describes the expected output structure, and the SDK enforces that structure on the model's response. The image URLs are passed directly to the model rather than the image bytes, since Cloudinary URLs are publicly readable and the model accepts them. The model returns a structured object containing summary, steps, key concepts, observations, and topics, all of which are persisted to the ai_explanation table.

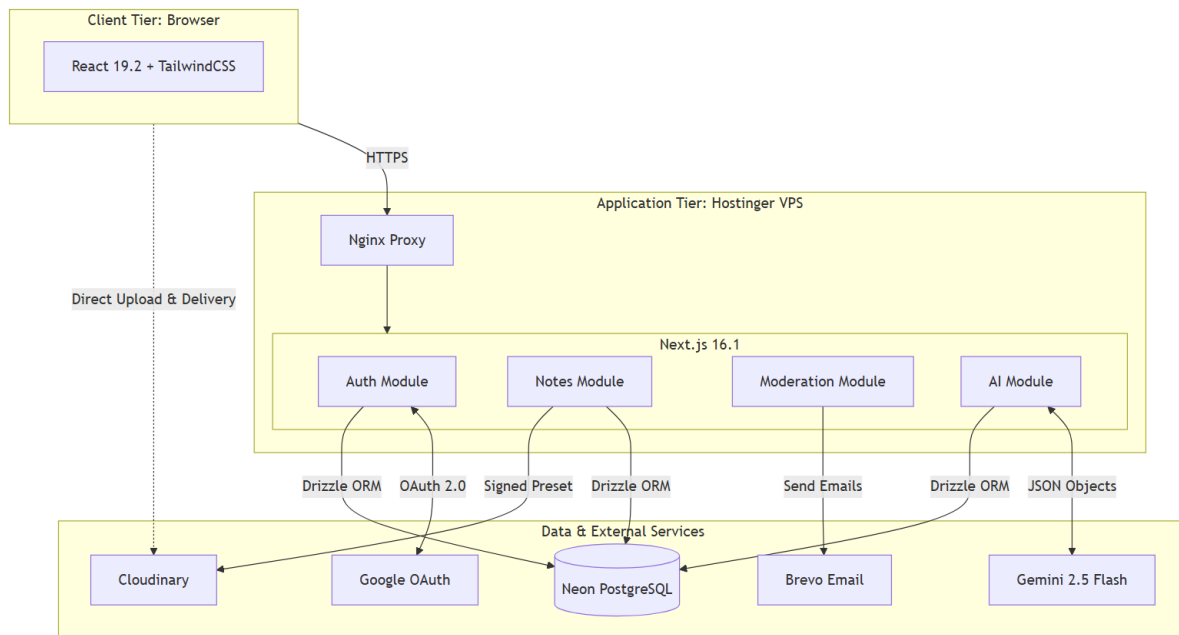


Figure 3.1: System Architecture of Prepify

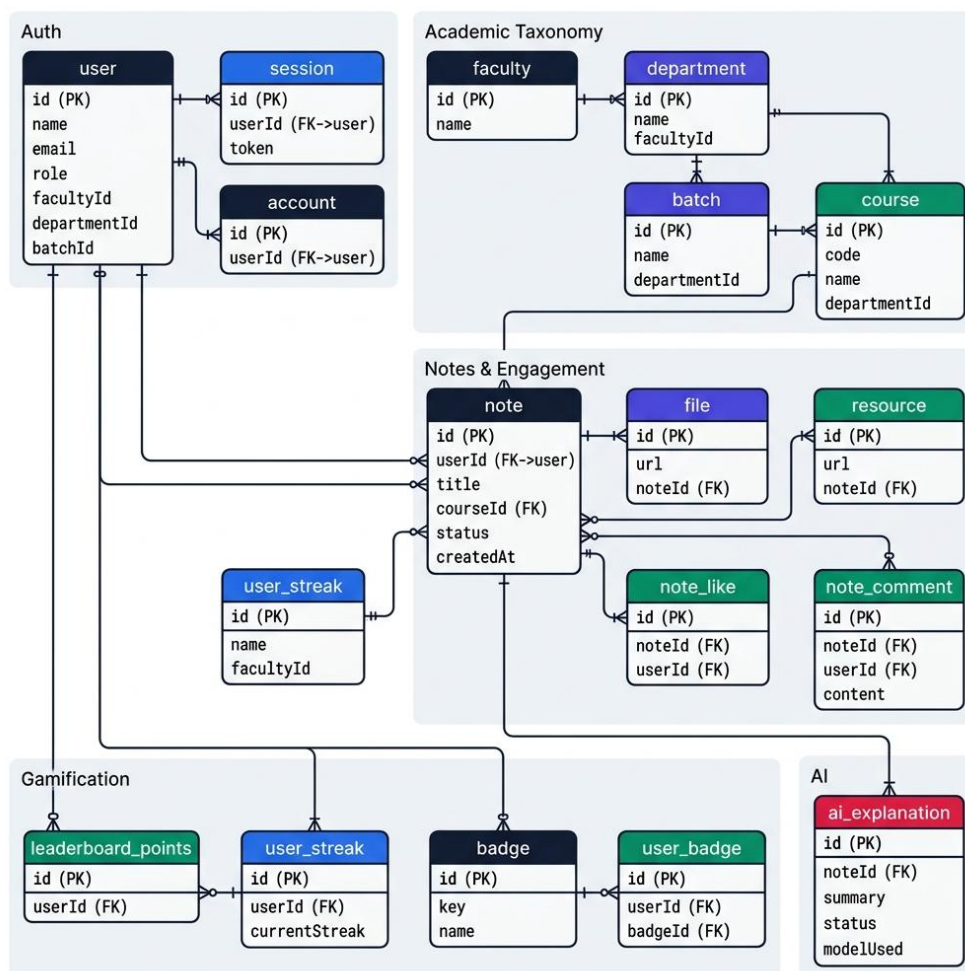


Figure 3.2: Entity Relationship Diagram

3.1.3 Functional and Nonfunctional Requirements

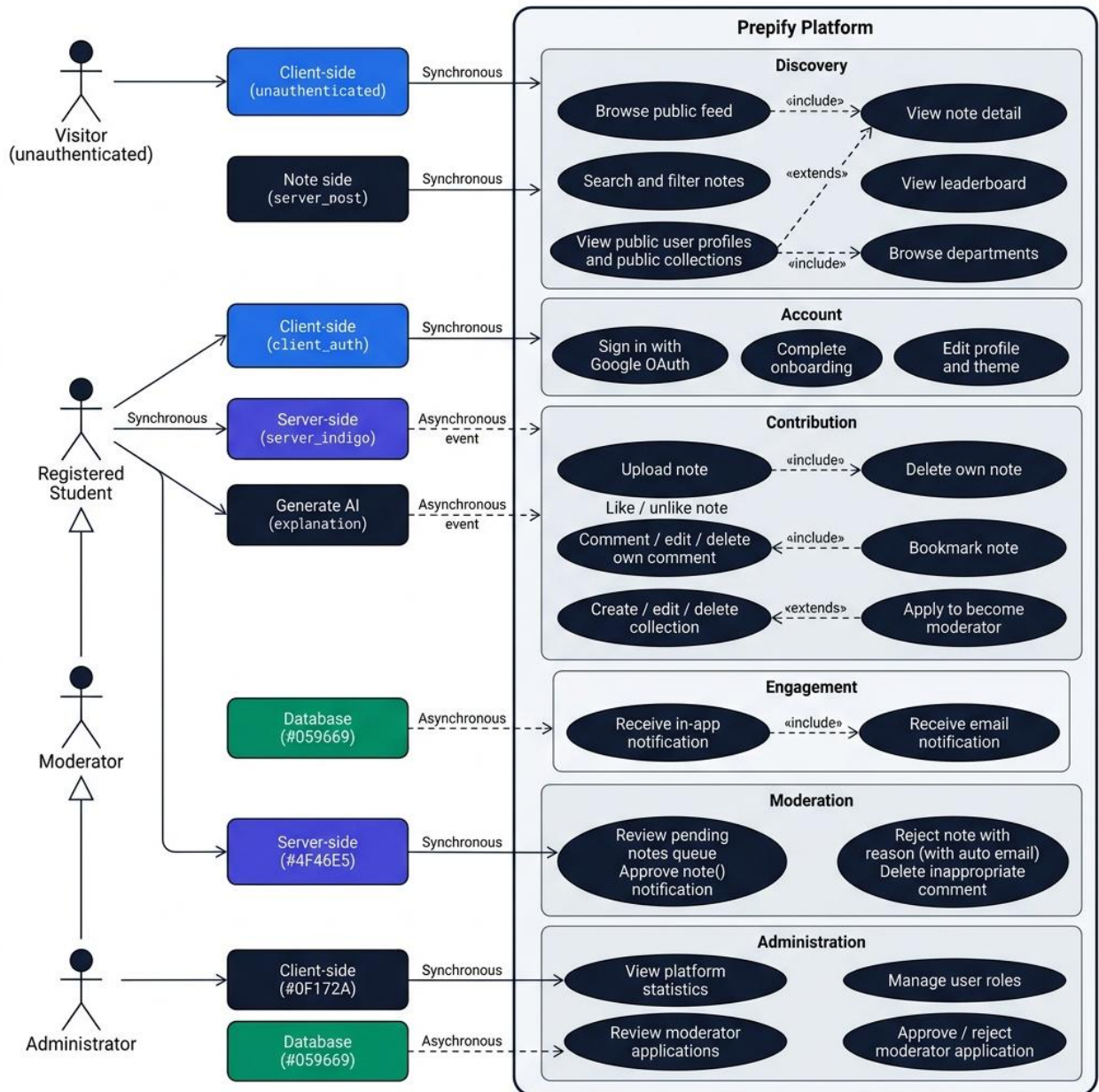


Figure 3.3: Use Case Diagram

Functional Requirements

The platform has been designed against a fixed set of functional requirements that emerged from the early observation phase. These describe what the system must allow its users to do, regardless of how it is implemented.

A user must be able to sign in through Google OAuth and be taken through an onboarding flow on first use that captures a unique username, the faculty they belong to, the department they study in, the batch they joined, and a preferred theme. Once onboarded, a user must be able to upload a note that has a title, an optional description, a course assignment, between one and a configurable number of images, and an optional list of reference URLs. Each uploaded note must enter a moderation queue and become publicly visible only after a moderator approves it.

A user must be able to like or unlike any approved note. Each like must increment the note's like count and add one transactional point to the uploader's leaderboard total. A user must be able to comment on any approved note, edit their own comments, and delete their own comments. The note's uploader must receive an in app notification and an email when their note is liked or commented on. A user must be able to follow any other user, in which case the follower must receive a notification when the followed user uploads a new note, and the followed user must receive an email when they gain a new follower.

A user must be able to bookmark any approved note. Bookmarks may be uncategorized or organized into named collections. Collections may be private or public. A user must be able to view their own bookmarks and collections from a dedicated page.

A user must be able to view a leaderboard ranking the most active contributors by their total points. The leaderboard must reflect changes in real time without manual refresh. The system must track each user's daily activity streak and award achievement badges when defined thresholds are crossed. Badges must be visible on the user's public profile. A moderator must be able to view a queue of pending notes, see the original images, and approve or reject each one. A rejection must include a reason that is then displayed to the uploader. An administrator must additionally be able to view platform wide statistics, manage user roles, approve or reject moderator applications, and remove inappropriate content.

The system must support the generation of an AI explanation for any approved note that has at least one image. The explanation must include a summary, a numbered list of steps, a list of key concepts, an observations field, and a list of topic tags. Each note must allow a maximum of three AI regenerations to discourage abuse.

The system must send transactional emails for follows, comments on the user's notes, approval or rejection of submitted notes, and points milestones.

Nonfunctional Requirements

The non functional requirements describe how the system performs and how it should behave under load and across different environments.

The platform must serve initial pages within two seconds on a standard broadband connection in Bangladesh. It must serve subsequent client side navigations in under five hundred milliseconds. The backend must handle at least one hundred concurrent users on the current production hardware without queueing or timing out. All client to server traffic must travel over HTTPS using TLS 1.3, and authentication tokens must never leave the secure session cookie.

The interface must be responsive across desktop, tablet, and mobile screens, with a minimum supported width of three hundred and seventy five pixels and no horizontal scrolling at any breakpoint. The platform must support a system aware dark mode in

addition to the default light mode.

Image uploads must accept JPEG, PNG, and WebP formats up to ten megabytes per file. Cloudinary must apply automatic format and quality optimizations on delivery so that downstream bandwidth use is minimized.

The codebase must be written in TypeScript with strict mode enabled, must lint cleanly under the ESLint Next.js recommended rules, and must use Drizzle's typed query builder for all database access. The system must maintain a target uptime of 99.5 percent on the Hostinger VPS, and must continue to serve cached images even if the application server is briefly restarted, since Cloudinary's CDN is independent of the application's lifecycle.

The AI explanation feature must enforce a regeneration cap of three runs per note to control cost and discourage misuse. AI calls must be wrapped in error handling so that a model failure marks the explanation record as failed rather than losing the existing data.

3.1.4 Data Flow Diagram

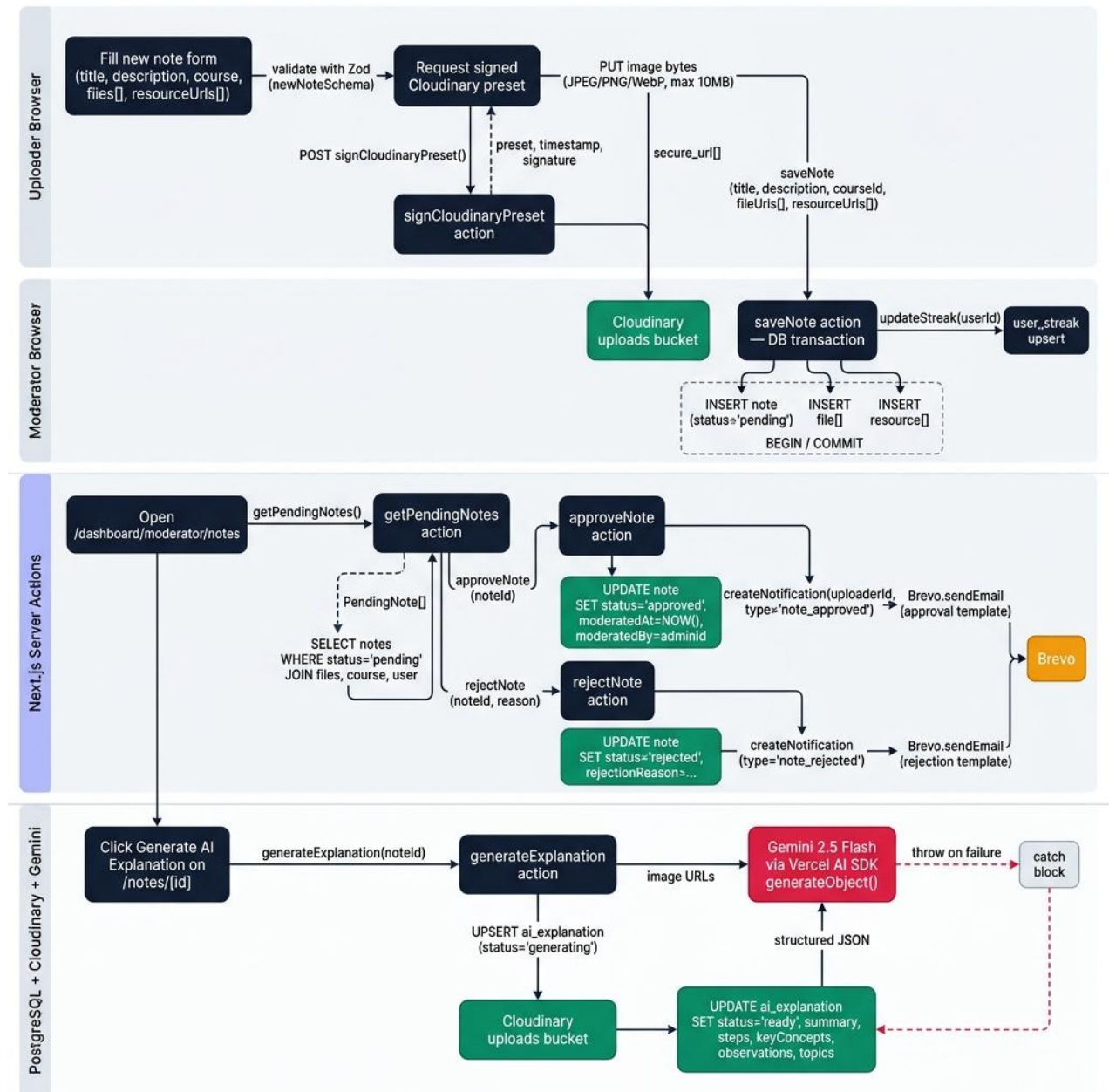


Figure 3.4: Data Flow Diagram

The data flow inside the platform follows a small number of well defined paths. A typical note upload starts at the browser, where the user fills the new note form and selects images. The form validates the inputs through a Zod schema, then calls a server action that signs a Cloudinary upload preset and returns it to the browser. The browser uploads each image directly to Cloudinary and gets back the URLs. It then calls a second server action with the title, description, course, image URLs, and optional reference links. That action writes a new row into the note table with status set to pending, writes the image URLs into the file table, writes the reference links into the resource table, and triggers the streak update for the user.

Once the note is in the queue, a moderator visits the moderator dashboard, which calls a server action that fetches all pending notes joined with their images and their uploader details. The moderator approves or rejects each item. Approval updates the status column to approved and sets moderatedAt and moderatedBy. The system then sends an in app notification and an email to the uploader telling them the note is live. Rejection requires a reason and follows the same path with a different status and a different message.

A like on an approved note is handled optimistically. The browser updates the like count immediately through a Zustand store, then calls a server action that wraps the database write inside a transaction. The transaction inserts a row into the note_like table (whose unique constraint on (noteId, userId) prevents duplicates), increments the leaderboard points for the uploader, and writes a leaderboard transaction row with the source set to note_like. If any step fails, the entire transaction rolls back and the optimistic update is reverted on the client.

The AI explanation flow starts from the note detail page. Only the uploader of an approved note sees the "Generate AI explanation" button. Clicking it calls the generateExplanation action, which checks the user's authentication, verifies that the requester is the note's uploader, confirms that the note is approved, checks that the regeneration count is below three, and inserts or updates an ai_explanation row with status generating. The action then collects the image URLs from the file table and passes them, along with the course name, into generateNoteExplanation. That helper builds a structured prompt, calls Gemini 2.5 Flash via generateObject, and returns a typed object validated against the Zod schema. The action writes the response back to the ai_explanation row and sets status to ready. If the AI call throws, the action catches the error and sets status to failed.

3.1.5 UI Design

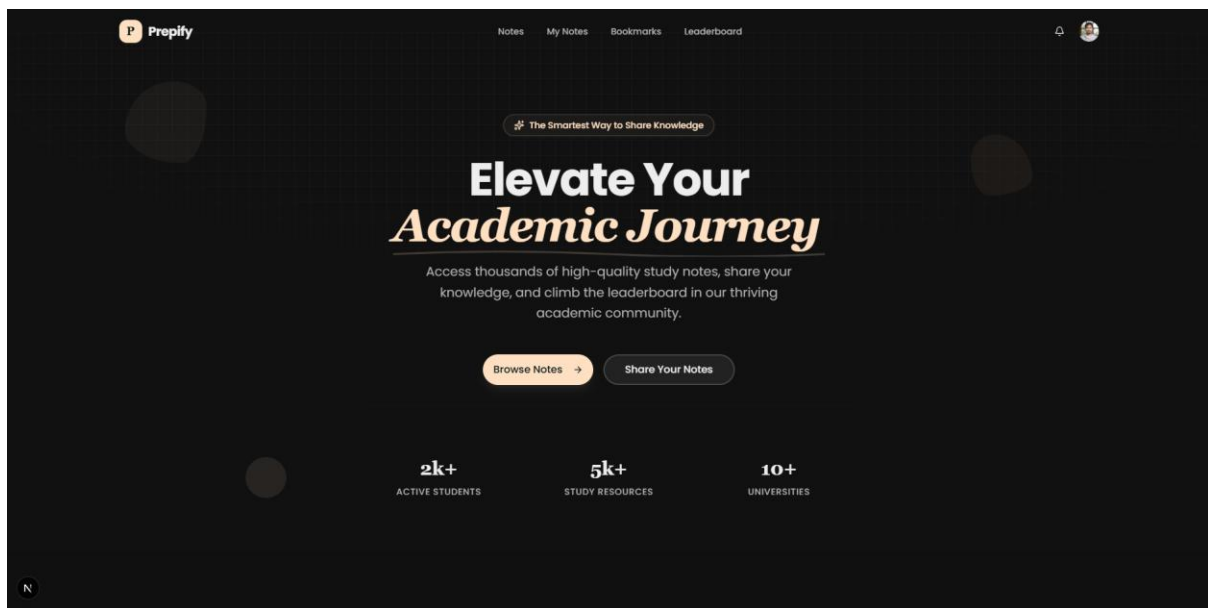


Figure 3.5: Landing page

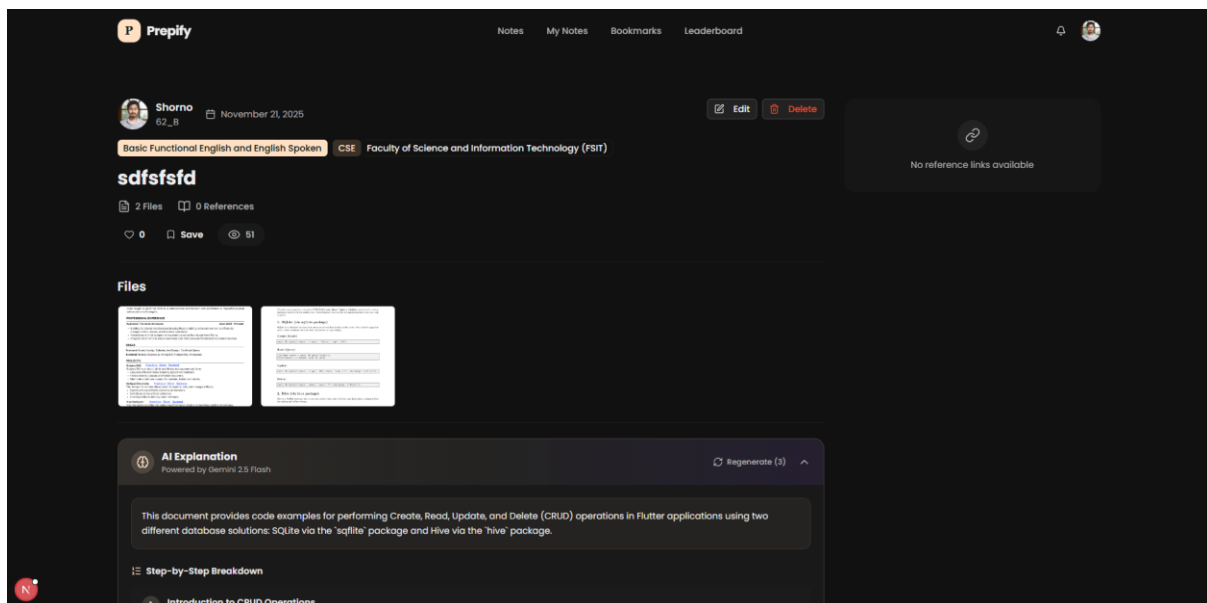


Figure 3.6: Note details page

Department: Computer Science & Engineering

Course: Select course

Title: e.g., Binary Search Tree — Insertion & Deletion Steps

Description (optional): 0/500

Upload Files

Drop your files here, or browse
Images & PDFs · Up to 10 files · 10MB each

AI Explanations — After approval, generate an AI-powered study guide from your notes

Reference Links (optional) + Add

Share Note

Figure 3.7: Note Upload page

Leaderboard
Top contributors in our community

3 Contributors 429 Total Points

RANK	USER	POINTS	NOTES	UPDATED
2	Cute Cinnury Kabir Rahy – 24 pts	24 pts	1	Nov 22
1	Shomo 389 pts	389 pts	11	Apr 25
3	Shomo 16 pts	16 pts	1	Nov 23

Figure 3.8: Leaderboard page

3.2 Detailed Methodology and Design

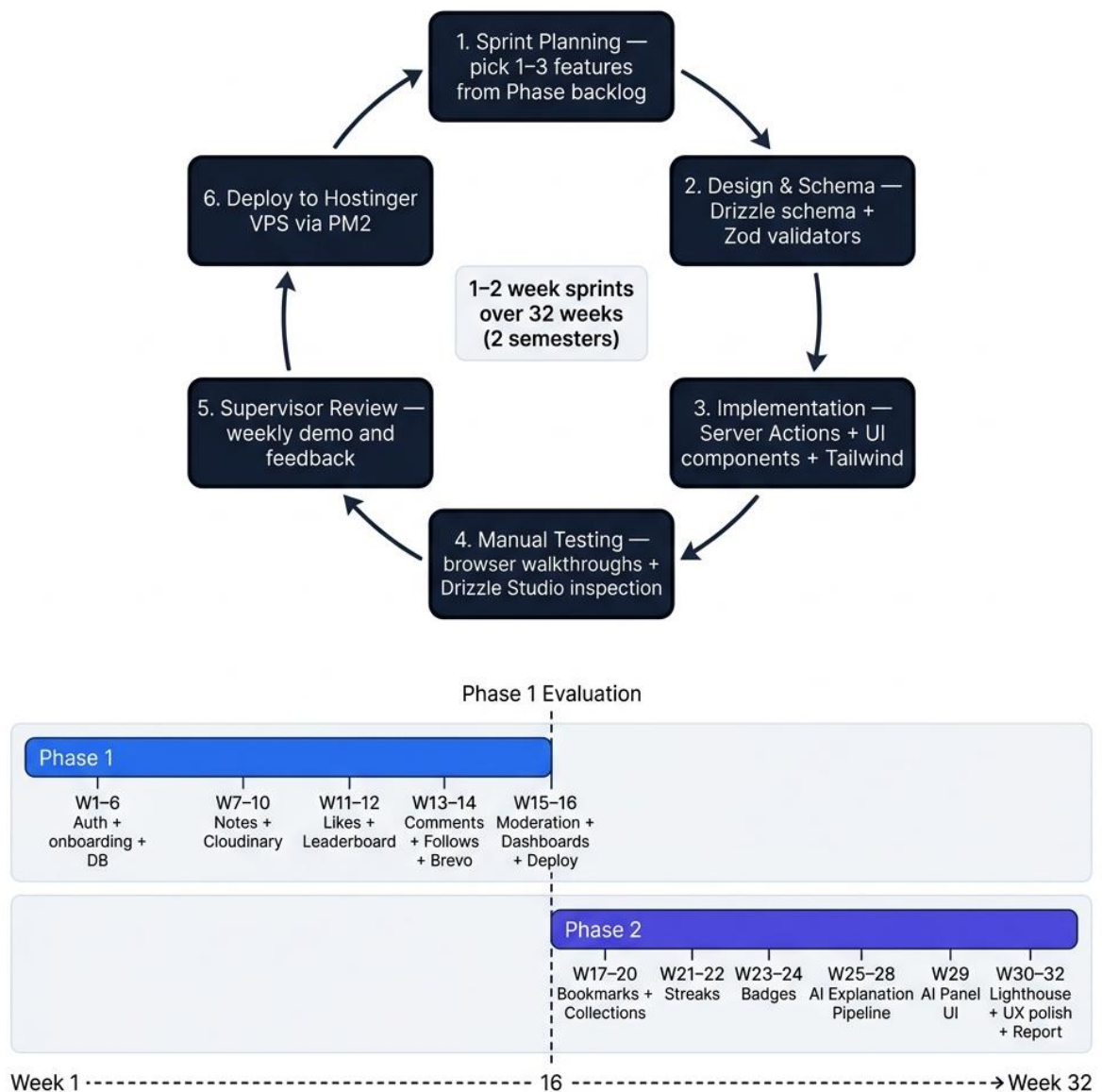


Figure 3.9: Agile Methodology Cycle + Phase Timeline

The design process consciously evaluated alternatives at every important decision point rather than picking the first option that came to mind.

For the frontend framework, three options were on the table. A traditional Create React App build would have produced a single page application served from a static host, which is the simplest deployment but ships everything to the client and has poor SEO for content that should be discoverable. Vue.js with Nuxt would have offered server side rendering similar to Next.js, but the AI SDK ecosystem and the Drizzle ORM tooling are noticeably less mature on the Vue side. Next.js with the App Router was chosen because it provides server side rendering, server components, server actions, image optimization, and route grouping in a single coherent system, and because the React 19 features integrate with it naturally.

For the database, three options were also considered. MongoDB would have offered schemaless storage with simple JSON-style documents, but the platform's data is deeply relational. A user has notes, notes have likes and comments and files and resources, each like generates a leaderboard transaction, each note has a course and a department. Flattening this into documents would have hidden the relations and made cascading deletes harder. Firebase would have provided real time syncing and authentication out of the box, but its data model is also document oriented and its pricing scales unpredictably with read counts. PostgreSQL on Neon was chosen because it handles the relational model directly, supports proper foreign key constraints, and Neon's serverless driver works well with Next.js Server Actions running on a VPS.

For the ORM, Prisma and Drizzle were both evaluated. Prisma generates a heavyweight client with its own query language and tends to be slow during development because of its code generation step. Drizzle is closer to SQL, has a smaller runtime, generates strict TypeScript types directly from the schema, and integrates cleanly with Neon's serverless driver. Drizzle was chosen on those grounds.

For authentication, NextAuth (now Auth.js), Firebase Auth, Clerk, and Better Auth were considered. NextAuth is the most established but its session model is somewhat opaque. Firebase Auth would have meant pulling in the Firebase SDK only for sign in. Clerk has a polished UI but charges per active user above a free tier. Better Auth was chosen because it integrates naturally with Drizzle, it stores sessions in the same Postgres database, and it allows fine grained control over the session shape.

For image storage, AWS S3 with CloudFront, Firebase Storage, and Cloudinary were the candidates. S3 plus CloudFront is powerful but requires writing image transformation logic separately. Firebase Storage shares the issue of pulling in the Firebase SDK for one purpose. Cloudinary was chosen because it includes signed presets, automatic format negotiation, and on the fly transformations on a single URL, and its free tier is generous enough for a student project.

For the AI provider, OpenAI's GPT 4o, Anthropic's Claude Sonnet, and Google's Gemini 2.5 Flash were all considered. Cost was the deciding factor. Gemini 2.5 Flash currently offers the lowest per request cost for multimodal tasks at this quality level, and it integrates with the Vercel AI SDK's generateObject function so that the structured Zod schema is enforced on the response. The provider abstraction is kept in a single file (lib/ai/provider.ts), which means switching providers is a single line change should the cost or quality balance shift.

For email, AWS SES and Resend were the alternatives to Brevo. Brevo was chosen because its free tier of three hundred emails per day is more than enough for the current usage and because its REST API is straightforward to call from a Next.js server action.

3.3 Project Plan

The project was split into two phases that mapped to the two FYDP semesters. The first phase ran from week 12 to week 30 and delivered the social platform layer. The second phase ran from week 30 to week 48 and delivered the AI explanation pipeline, the bookmarks and collections subsystem, the streaks and badges system, and a series of UX refinements.

In Phase 1, the project began with infrastructure setup. The Next.js project was created, the database was provisioned on Neon, the Drizzle migrations were generated, and the first few entities (user, faculty, department, course, note) were defined. Authentication came next, with Better Auth wired to Google OAuth and the onboarding flow built around the new user creation hook. The note upload feature followed, with the Cloudinary integration done first because it gates everything else. The voting and commenting features were then added in parallel, since both follow a similar server action plus optimistic UI pattern. Follows and notifications came next, with Brevo added at the same time so that the email side could be tested end to end. The moderation system, the admin dashboard, and the moderator dashboard rounded out Phase 1.

In Phase 2, the bookmarks and collections were built first because they share the same engagement pattern as likes. The streaks system was added next, with a single server action that runs on every meaningful interaction and updates the daily streak record. The badges system was layered on top, with seeded definitions and a check action that runs after each upload, like, comment, or follow. The AI explanation feature was the largest single piece of Phase 2: the schema, the Zod validation, the provider abstraction, the prompt design, the regeneration cap, and the panel UI all went in across several sprints. The phase closed with a series of polish passes covering accessibility, dark mode contrast, responsive layouts on small phones, and Lighthouse score improvements.

3.4 Task Allocation

Since this is a single student final year design project, the author has been responsible for every task. The supervisor and co supervisor have provided regular reviews, suggested feature additions, and reviewed the report drafts, but the implementation work has been done end to end by the student. The timeline below lays out the principal activities week by week.

Table 3.1: Project Timeline

Tasks	Weeks																			
	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	
Schema design, Auth setup																				
Note upload flow.																				
Moderation and gamification																				
AI pipeline.																				

3.5 Summary

This chapter laid out the design and the methodology behind the platform. The system uses a three tier architecture built on Next.js, PostgreSQL through Drizzle, Cloudinary, and Gemini 2.5 Flash. The functional requirements describe what users must be able to do, and the non functional requirements describe how the system must perform under load and across devices. The data flow shows how a note moves from upload through moderation to AI explanation, and the UI design covers the look and feel across devices and roles. Several alternatives were considered at every important decision point, with the final stack chosen on the basis of fit, cost, and maintainability. The project plan and the task allocation describe how the work was scheduled across two FYDP semesters. The next chapter moves to implementation, describes how each module was actually built, and presents the results obtained from the running platform.

Chapter 4

Implementation and Results

This chapter describes how the platform was implemented in practice. It walks through the development environment, the major modules that make up the application, the testing approach used, and the results observed on the production deployment.

4.1 Environment Setup

Development took place on a Windows 11 machine running WebStorm as the primary IDE. Node.js 20 LTS provided the JavaScript runtime, and pnpm was used as the package manager because of its strict node module resolution and disk efficient store. The project sources are kept in a single Git repository hosted on GitHub.

The database runs on Neon, a serverless Postgres provider that supports a connection pooling proxy and a serverless driver designed for short lived workloads. During development, the Next.js development server is started with Turbopack, which gives near instant hot module replacement even when Drizzle's generated types change. Drizzle Kit handles migration generation through drizzle-kit generate and applies them through drizzle-kit push. Drizzle Studio is used to inspect and edit rows directly during development.

Image hosting goes through Cloudinary. A signed upload preset is configured in the Cloudinary dashboard and referenced by the server action that signs uploads on behalf of the browser. Email goes through Brevo, with a single API key stored in the environment.

The production deployment runs on a Hostinger VPS with two virtual CPUs, eight gigabytes of RAM, and a one hundred gigabyte NVMe SSD. The operating system is Ubuntu 22.04 LTS. The Next.js production build is launched through PM2, which keeps it alive across crashes and reboots. Nginx sits in front of the Next.js process as a reverse proxy and terminates TLS using a Let's Encrypt certificate. The custom domain prepify.space was registered for one year and points to the VPS through an A record.

Chrome DevTools and React DevTools were used heavily during debugging, particularly the React component tree inspector and the network tab for verifying optimistic updates. Lighthouse runs on the production deployment were used to measure performance and accessibility. The browser based Drizzle Studio was used to inspect the live database while debugging server actions.

Implementation

The following subsections walk through each major module of Prepify in the order that they were built.

4.2.1 Authentication and Onboarding

Authentication was the first feature to be wired up because everything else depends on it. Better Auth is configured with Google OAuth as the only sign in method. When a user clicks the sign in button on the login page, the application redirects them to Google's OAuth consent screen with the appropriate client ID and scopes. After consent, Google returns an authorization code, which Better Auth exchanges for the user's profile data, including name, email, and profile picture URL. A session row is written to the session table, and a secure HTTP only session cookie is set on the browser.

On first sign in, a row is created in the user table without a username, faculty, department, or batch. The route group's layout detects this incomplete state and redirects the user to /get-started, which hosts a multi step onboarding form. The form is split into screens for username selection, role information, faculty and department selection (with cascading dropdowns), batch selection, and theme preference. Each step is validated with Zod, and the username step also runs a server action that checks for uniqueness in the database. Once all steps are complete, a single server action writes the captured fields back to the user row and the user is redirected to the main feed.

Session persistence has been verified across page reloads, browser restarts, and across tabs. Edge cases such as a revoked Google permission or an expired token are handled gracefully by clearing the session and redirecting to the login page.

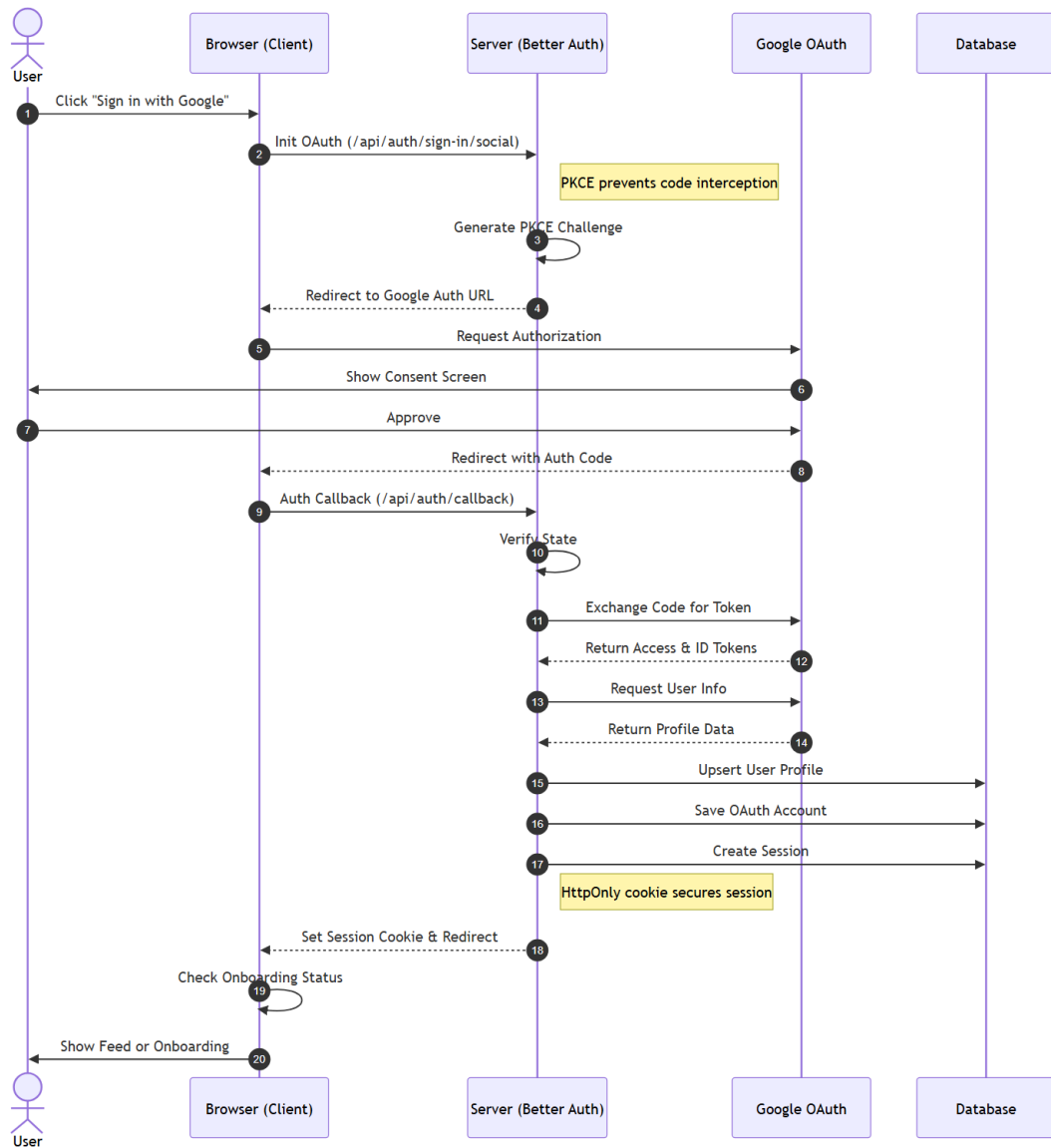


Figure 4.1: Authentication Flow

4.2.2 Note Upload Module

The note upload flow has two halves. The first half runs entirely on the browser. The form, built with React Hook Form, captures the title, description, course, images, and optional resource URLs. Image previews are rendered immediately using the browser's `URL.createObjectURL`. Each image is then uploaded directly to Cloudinary using a signed upload preset that is requested from a server action. The server action signs the preset using the Cloudinary API secret and returns the signature, the timestamp, and the upload preset to the browser, which then makes the upload request itself. This avoids the application server ever touching the file body.

The second half runs on the server. After all uploads succeed, the browser calls the `saveNote` server action with the form data and the array of returned Cloudinary URLs. The action inserts a row into the note table with status set to pending, then bulk inserts the corresponding rows into the file table and the resource table inside the same transaction. The transaction approach makes sure that a partial failure does not leave a note without its files. Once the transaction commits, the action triggers the streak update for the uploader and revalidates the relevant cache tags so that the My Notes page reflects the new submission immediately.

The form supports up to ten images per note and accepts JPEG, PNG, and WebP up to ten megabytes per file. Larger files are rejected on the client side before the upload starts.

4.2.3 Voting and Leaderboard

The voting subsystem was built around the like only model rather than upvote and downvote. The decision came from observing that downvotes on academic content tend to be punitive rather than informative and discourage submission. A single like is enough to surface the most useful notes through the sort order of the feed.

When a user clicks the heart icon on a note card, the client side Zustand store updates the like count optimistically and toggles the heart state. In parallel, the `likeNote` server action runs on the server. The action wraps three database writes inside a single transaction: insert a row into `note_like`, increment the leaderboard points for the uploader, and write a row into the leaderboard transaction log with the source set to `note_like`. The unique constraint on (`noteId`, `userId`) in the `note_like` table prevents duplicate likes even if the action is called twice in quick succession. If the transaction fails, the action returns an error and the client reverts the optimistic update.

The leaderboard itself is computed by reading the `leaderboard_points` table, joined with the user table to fetch usernames and avatars. The query is ordered by total points descending and limited to the top contributors. Because every change to a user's points is also recorded in the transaction log, it is straightforward to audit any individual point and trace it back to the like, comment, or other event that produced it.

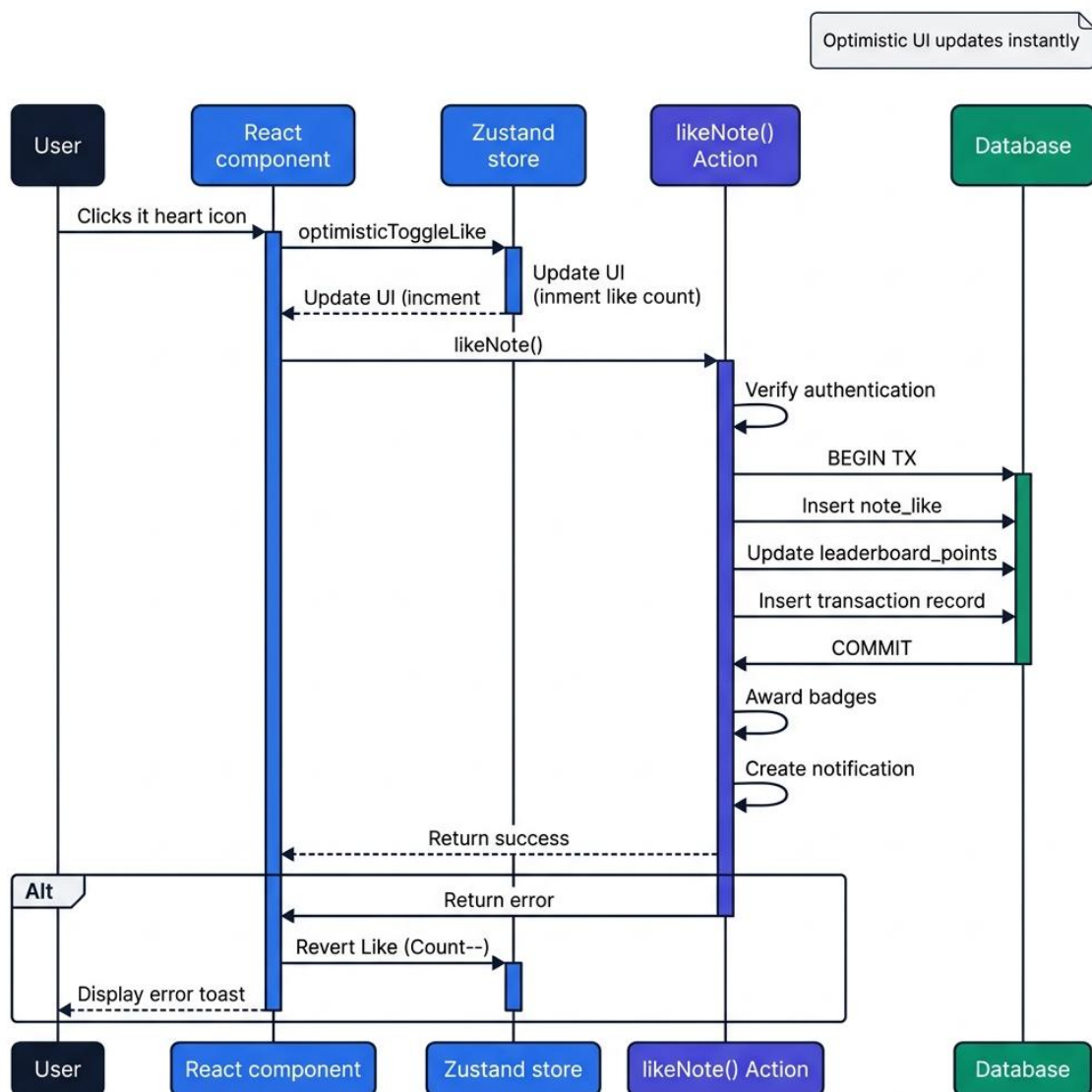


Figure 4.2: Like and Leaderboard Transaction Flow

4.2.4 Comments and Follows

Comments are stored in the `note_comment` table with a foreign key to the note and to the author. The create, edit, and delete actions all run as server actions and apply authorization checks based on the session user. The author can edit and delete their own comments. Moderators and administrators can delete any comment. The author of the note receives an in app notification when a new comment is posted on their note and an email through Brevo.

Follows are stored in the `follow` table with foreign keys to the follower and the followed user, plus a unique constraint to prevent duplicate follows. The follow and unfollow actions both run inside transactions so that the notification side effect is consistent with the relation change. When user A follows user B, B receives an in app notification and an email. When B uploads a new approved note, all of B's followers receive a new note notification.

4.2.5 Bookmarks and Collections

The bookmarks feature is built around two tables. The collection table holds named collections with a public or private flag and an optional description. The bookmark table holds a single bookmark, optionally linked to a collection. A bookmark without a collection ID is treated as belonging to the default uncategorized list. The unique constraint on (`userId`, `noteId`) in the bookmark table keeps the same note from being bookmarked twice by the same user.

The bookmark icon on a note card calls the `toggleBookmark` action, which either inserts or deletes the row depending on the current state. The collection picker is a popover that lists the user's existing collections plus an option to create a new one. New collections are created through the `createCollection` action, which validates the name length, the public flag, and the optional description.

A public collection is discoverable on the public profile of its owner. This turns Prepify into a curation platform on top of the note sharing platform. A senior who is preparing other students for an exam can build a collection of community ranked notes for that exam, mark it public, and share the URL.

4.2.6 Streaks and Achievement Badges

The streaks system was added because competing platforms either ignore engagement entirely or rely on raw counts. A streak captures continuity of engagement, which is a much more useful signal for both the user and the platform.

The `user_streak` table stores the current streak, the longest streak the user has ever achieved, and the date of the user's last activity stored as a YYYY-MM-DD string. The `updateStreak` action runs on every meaningful interaction, namely a new note upload, a like, a comment, a new follow, or a successful AI explanation generation. It compares the user's last active date with the current date. If they are equal, nothing changes. If the current date is exactly one day after the last active date, the current streak is incremented and the longest streak is updated if the new value is greater. If the gap is more than one day, the current streak resets to one. The action is idempotent within the same day, which means hitting it ten times on the same day has the same effect as hitting it once.

The badges system is a pair of tables. The badge table stores the definition of each badge, with a key, a name, a description, an icon name, a category (upload, streak, engagement, social), and a numeric threshold. The seed file

populates this table with a curated set of badges such as "First Upload", "Ten Notes", "Hundred Likes Received", "Seven Day Streak", "Helpful Commenter", "Resource Linker", and "Department Champion". The `user_badge` table is the junction table that records which badges a user has earned, with a unique constraint on (`userId`, `badgeId`) so that a badge cannot be awarded twice.

After every meaningful interaction, the `checkAndAwardBadges` action computes the user's current totals (uploads, likes received, comments written, follows, current streak) and inserts rows into `user_badge` for any badge whose threshold has just been crossed. The action ignores badges that the user already holds. New badges are then displayed on the user's public profile.

4.2.7 Moderation, Admin Dashboard, and Moderator Applications

Every uploaded note enters the system in the pending state. The note is visible to its author on the My Notes page with a clear status badge, but it does not appear on the public feed until a moderator approves it.

Moderators access the moderator dashboard through the `/dashboard/moderator` route group. The route layout checks the session user's role and rejects anyone who is not a moderator or an administrator. The main moderator page is a queue of pending notes joined with their files, course, and uploader, ordered by the oldest pending note first. Each row in the queue exposes an approve button and a reject button. Approve writes `status = approved`, sets `moderatedAt` and `moderatedBy`, and triggers the email to the uploader. Reject opens a small modal that asks for a reason, which is then stored on the note row and shown on the uploader's My Notes page.

The administrator dashboard is one level above this and is reached through `/dashboard/admin`. The admin landing page shows total registered users, total approved notes, total pending notes, and a list of recent moderation events. The user management page lists every user with their current role and exposes a control to promote or demote between user and moderator. The moderator applications page lists all pending applications, each with the applicant's username, the reason they typed when applying, and approve or reject controls.

The moderator application flow itself is exposed to ordinary users through the `/become-moderator` route. The form captures a short statement of intent, validates it through Zod, and writes the application to the `moderator_application` table with status pending. The administrator then reviews the application and either promotes the user to moderator or rejects the application with a reason.

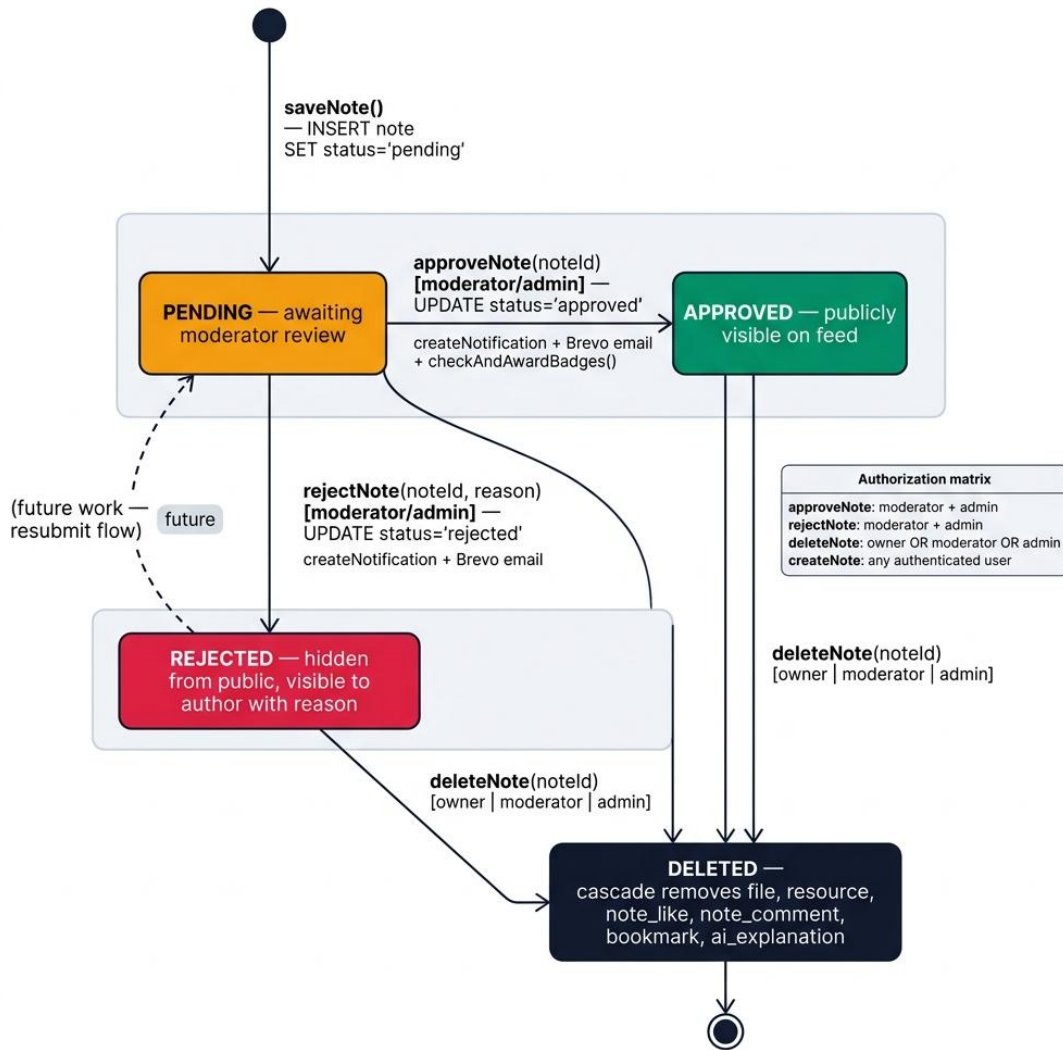


Figure 4.3: Note Moderation State Machine

4.2.8 Notifications

Notifications are delivered both in app and through email. The notification table stores in app notifications with a type, a recipient user ID, the actor user ID, an optional note ID, and a read flag. A header bell icon on every authenticated page fetches the count of unread notifications and renders a small popover with the most recent ones. Marking a notification as read updates the row, and a "mark all read" button updates them in bulk. Email notifications are sent through Brevo's transactional email API. The send is wrapped in a try / catch that logs failures but does not block the originating action, since a missed email should not roll back a successful follow or comment.

4.2.9 AI Explanation Pipeline

The AI explanation feature is the centrepiece of Phase 2 and the most distinctive part of Prepify. The goal is simple to describe and reasonably hard to deliver well: take the photographed pages of a handwritten note, send them to a multimodal large language model, and persist a structured explanation that students can read alongside the original. The first design decision was whether to use a separate OCR step. The traditional pipeline would extract text using Tesseract or a similar engine, send the text to a text only LLM, and assemble the explanation from the LLM's response. This has two problems. First, OCR on irregular handwriting tends to lose layout information such as which line is part of which equation, which is exactly the information a tutor needs to explain a solution. Second, modern multimodal models like Gemini 2.5 Flash can read images directly, and they keep the spatial structure intact. Sending the images straight to the model produces a noticeably better explanation. The platform therefore skips the OCR step entirely.

The second design decision was how to enforce the output structure. A free form completion would require the application to parse the response and would break unpredictably on any small variation. The Vercel AI SDK's `generateObject` function takes a Zod schema, prompts the model with both the user message and the schema, and returns a typed object that matches the schema. The schema for Prepify's explanation is defined in `lib/ai/schemas.ts` and contains the summary, an array of steps with step number, title, and content, an array of key concepts with name and description, an optional observations field, and an array of topic tags. The SDK rejects any response that does not match the schema.

The third design decision was the prompt design. The system message tells the model that it is an academic tutor specializing in handwritten student work. It explains that the images are sequential pages of one cohesive solution from a specific course and that the content may be in English, Bangla, or a mix of both. It instructs the model to analyze all images together as one solution, identify each step of the solution process, list the key formulas or theorems, note any potential errors or alternative approaches, and generate three to eight topic tags. The user message simply asks the model to analyze the pages and provide a structured explanation. This split between system instructions and user request keeps the model's behaviour predictable across different note types.

The action itself, `generateExplanation`, lives in `actions/ai/generate-explanation.ts`. It first verifies the user's session, then verifies that the user is the uploader of the note in question, then verifies that the note is approved, then verifies that the note has at least one file, and finally checks the regeneration count against the maximum of three. If any of these checks fails, the action returns a structured error with the appropriate HTTP status code. If all the checks pass, the action either inserts a new `ai_explanation` row with

status generating or updates the existing row with an incremented regenerateCount. It then collects the image URLs from the file table, calls generateNoteExplanation with the URLs and the course name, and writes the resulting structured object back to the row with status ready. If the call throws, the action catches the error and updates the row to status failed.

The model call itself goes through the Vercel AI SDK. The provider is configured in a single line in lib/ai/provider.ts, which keeps it easy to switch providers in the future. The SDK currently sends the URLs of the Cloudinary images as URL objects rather than fetching the bytes manually, which works because Cloudinary serves the images on a public CDN.

The AI explanation panel on the note detail page renders the structured response in a clean reading layout. The summary appears at the top in a slightly larger font. The steps appear as a numbered list of cards, each with a title and a body. The key concepts appear as a definition list. The observations appear in a smaller note styled block. The topic tags appear as pill chips at the bottom. A footer line shows which model generated the explanation and how many regenerations remain out of the maximum three.

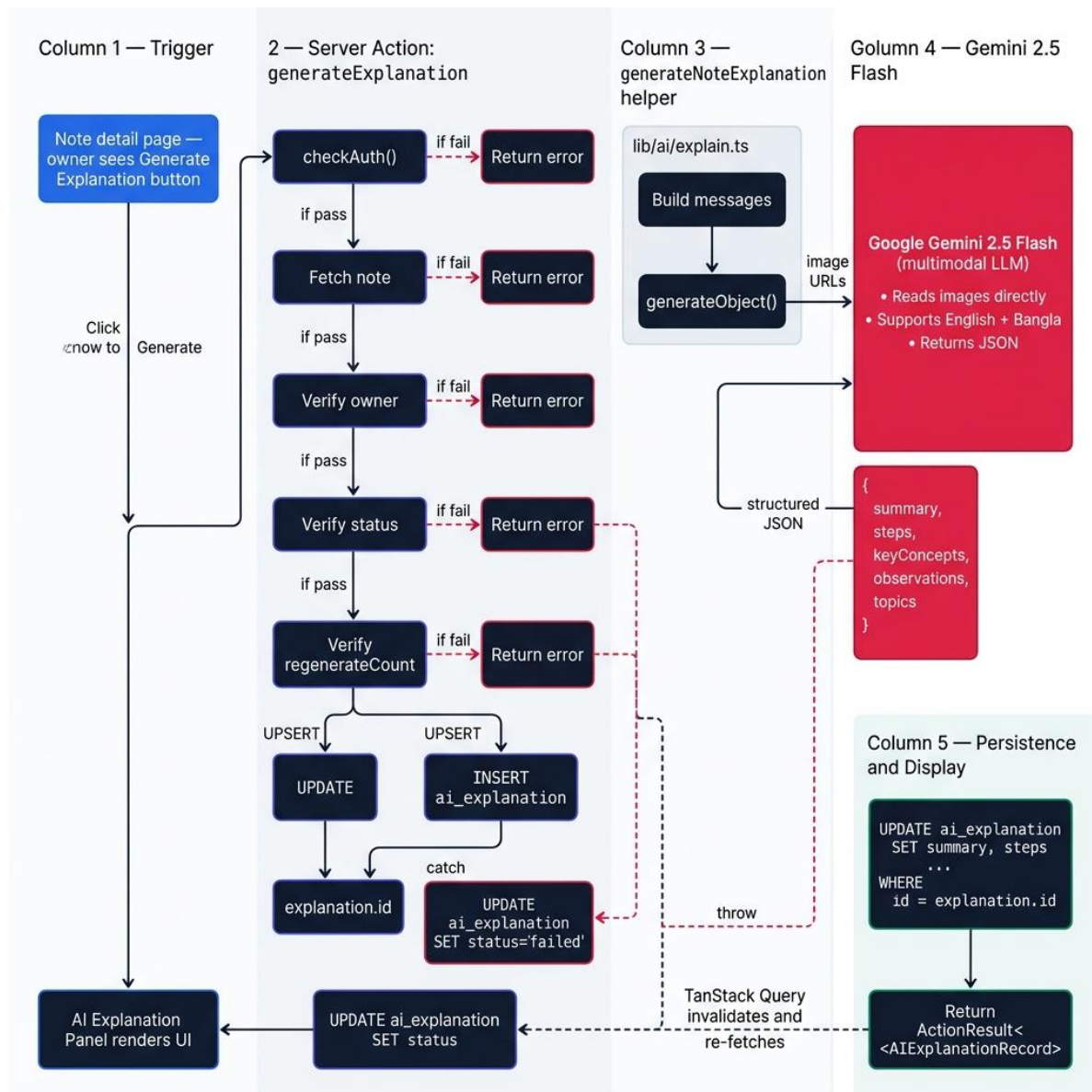


Figure 4.4: AI Explanation Pipeline

4.2.10 Home Page, Navigation, and Profile Pages

The home page is the front door of the platform. It works for unauthenticated visitors and showcases what the platform offers. The hero section presents a short tagline, a call to action to sign in, and a stats strip showing total users, total approved notes, and total likes given. The featured notes section pulls the highest liked approved notes from the last seven days. The how it works section walks new users through upload, approval, and discovery in three short cards. The department browser shows each department with the count of approved notes filed under it, with each card linking to a filtered notes feed.

Navigation is handled through the App Router's nested layouts. The route groups (app), (dashboard), (auth), and (onboard) each have their own layout. The (app) layout adds the main navigation bar, the notification bell, the search bar, and the user menu. The (dashboard) layout adds the role gated sidebar. The (auth) layout is minimal and used only for the login page. The (onboard) layout removes the main navigation entirely so that the onboarding flow takes the full screen.

The profile pages exist in two flavours. The public profile at `/user/[username]` shows the user's avatar, username, faculty, department, batch, total points, badges, and a paginated list of their approved notes. The private profile at `/profile/[userId]` is reserved for the owner and additionally shows pending and rejected notes, the bookmarks tab with collections, and links to settings.

4.2.11 Responsive Design and Dark Mode

The interface adapts cleanly from a 375 pixel mobile width up to a 1920 pixel desktop. Tailwind CSS 4 handles the breakpoint logic through its responsive prefix system. Components from shadcn/ui have been adjusted where needed to keep tap targets large enough on mobile and text readable on small screens.

Dark mode is implemented through next-themes. The theme toggle is exposed in the user menu and during onboarding. The default behaviour follows the operating system preference. The dark mode color tokens are designed for comfortable reading on OLED screens, with a near black background and a soft slate text color.

4.2 Testing and Evaluation

Testing for the platform was carried out entirely by hand. No automated test suite was written for the project, which means there are no unit tests, no integration tests, and no end to end browser automation scripts in the codebase. The decision to skip automation was a deliberate one given the time budget of a single student project across two semesters. Setting up a Jest or Vitest harness alongside the Server Actions, mocking Cloudinary and the Gemini API, and maintaining a Playwright suite for the UI flows would have consumed several weeks of work that would otherwise have gone into building features. In place of that, every feature was validated through manual walkthroughs in the browser, and the database state was inspected through Drizzle Studio after each action to confirm that the writes had

landed correctly.

Each major user flow was written down as a short test case in a spreadsheet, with preconditions, the exact steps to follow in the browser, and the expected outcome. Each case was then executed by hand with several different test accounts representing the regular user, moderator, and administrator roles. When a test failed, the issue was reproduced in the browser and traced through the running code; once the fix was in place, the same case was re run by hand to confirm the fix.

The authentication flow was checked by signing in with three separate Google accounts. Session persistence was verified across page reloads, browser restarts, and across tabs. Onboarding was checked by signing in with a new account each time and walking through the username, role, academic info, and theme steps in the browser. Username uniqueness was verified by trying to claim an already taken username, which correctly returned an error message and prevented submission.

Note upload was checked by submitting single image and multi image notes through the upload form, using JPEG, PNG, and WebP files, with files at the size limit and just above the size limit, and with attached resource URLs. Each successful upload was confirmed both on the My Notes page (with status pending) and in Drizzle Studio. Cloudinary was confirmed to be receiving the images, applying the configured transformations, and serving them through its CDN with WebP delivery on supporting browsers.

The voting system was verified by liking a note from one account, then opening Drizzle Studio to confirm that the note's like count had incremented, that the uploader's leaderboard total had incremented, and that a transaction row had been written with source note_like. Liking the same note again from the same account was confirmed to leave the count unchanged. Unliking was confirmed to reverse all of the above.

The moderation workflow was checked end to end through the browser. A note uploaded from a regular account appeared in the moderator queue. The moderator account approved it, the note appeared on the public feed, and the uploader received both an in app notification and an email. A separate note was rejected with a reason; the reason appeared on the uploader's My Notes page and the public feed remained unchanged.

The follow and notification system was checked by having one account follow another. The followed account received an in app notification and an email. The followed account then uploaded an approved note, which produced a notification for the follower. Comments were checked by creating, editing, and deleting comments

from each role, with the authorization checks confirmed by attempting to edit another user's comment from the wrong account, which the server action correctly rejected.

Bookmarks and collections were checked by bookmarking a note into a private collection, into a public collection, and outside any collection. The public collection was confirmed visible on the user's public profile when opened from a different browser. Removing a collection was confirmed in Drizzle Studio to set the bookmarks' collection ID to null without deleting the bookmarks themselves.

Streaks were checked by uploading a note on day one and confirming through Drizzle Studio that the current streak became one. A second upload on day two pushed the streak to two. Skipping day three and uploading on day four reset the current streak to one and left the longest streak at two. To avoid waiting real days for some of the streak edge cases, the system clock on the development machine was temporarily moved forward to simulate the day boundary. Badges were checked by reaching each threshold by hand and confirming the badge appeared on the profile.

The AI explanation feature was checked with several handwritten notes covering different subjects (mathematics, computer science theory, and physics) and different image counts. The structured response was confirmed to match the Zod schema in every case by reading the stored row in Drizzle Studio. Regeneration was checked by clicking the regenerate button three times on the same note and then attempting a fourth time, which correctly returned a 429 status with the expected error message in the toast. The error path was checked by temporarily breaking the API key, which correctly produced a failed status on the explanation row.

Responsive design was checked across Chrome, Firefox, and Safari on desktop, and on Android devices with screen widths between 5.5 and 6.7 inches. Chrome DevTools' device toolbar was used for the additional viewport widths that were not directly available on a physical device. Layouts were verified to adapt without horizontal scrolling at any tested width. Dark mode was checked by toggling the OS theme and confirming that the platform followed the system preference when the user had not explicitly set a theme, and held the explicit choice when one was set.

4.3 Results and Discussion

The implementation successfully delivers all features described in the project outcome section of Chapter 1. The production deployment includes working support for sign in, onboarding, note uploads, voting, comments, user follows, notifications, bookmarks, collections, streaks, badges, moderation, the admin dashboard, moderator applications, AI explanations, responsive design, and dark mode.

The platform behaves predictably under normal usage. The optimistic UI for likes and bookmarks gives the impression of instant response, even though the database write happens a few hundred milliseconds later. The transaction wrapping around point updates means the leaderboard never drifts out of sync with the underlying like data. The Brevo integration delivers email reliably within seconds of the triggering event, which is fast enough for a notification email.

The AI explanation feature has been the most interesting result. On the test set of handwritten notes used during development, Gemini 2.5 Flash produced clean, well structured explanations on the first try around eighty per cent of the time. The remaining cases were mostly notes with very faint pencil writing or with text that was partially cut off by the camera frame. Even in those cases, the model gracefully degraded by noting which parts of the page it could not read, rather than fabricating content. The regeneration cap of three has so far been more than enough to recover from the few cases where the first response was unsatisfactory.

The leaderboard has worked as intended in keeping motivation high during testing. The transaction log makes it easy to verify that every point on the leaderboard has a corresponding event behind it. The badges have produced a small but real incentive for users to keep returning, and the streaks have produced a stronger one. Public collections have begun to behave like curated playlists during testing, with one test account preparing a "CSE 313 Final Prep" collection that pulled together notes from several different uploaders.

The platform's footprint stays well inside the free tiers of every supporting service at the current scale. Neon's free database has used about 100 megabytes of storage out of the available 500. Cloudinary has used about one gigabyte of its 25 gigabyte allowance. Brevo's daily free email limit has not been hit during normal use. The per request cost of Gemini 2.5 Flash is low enough that even active AI explanation use has not produced a bill above a few US dollars per month.

The chief bottleneck observed during testing was image upload latency on slow connections. Cloudinary's signed direct upload mechanism keeps the application server out of the upload path, but the upload itself still depends on the client's connection. On a slow mobile connection, uploading five large images can take longer than the model takes to generate the explanation afterwards. This is an unavoidable cost of supporting handwritten content on a free tier.

4.4 Summary

This chapter described the development environment, walked through every major module of the platform, presented the manual testing approach used in place of an automated test suite, and discussed the results from the running deployment. The implementation has produced a complete, stable platform that delivers everything promised in the project outcome section, including the AI explanation feature that distinguishes it from every comparable platform in the region. The next chapter steps back and looks at the engineering standards that the project complies with, the impact it has on its users and on the wider environment, the financial picture, and the mapping to complex engineering problem categories.

Chapter 5

Engineering Standards and Design Challenges

This chapter covers the standards that the project follows, the impact it has on society and the environment, the cost of running and maintaining it, and the way it maps to the complex engineering problem and engineering activities frameworks used by the FYDP committee.

5.1 Compliance with the Standards

Prepify follows a set of standards across software development, communication protocols, and security. Each standard was chosen after considering the alternatives, and the rationale is recorded below.

5.1.1 Software Standards

The codebase is written entirely in TypeScript with strict mode enabled in `tsconfig.json`. This forces every variable, parameter, and return type to be inferable, which catches a large number of bugs at compile time rather than at runtime. The JavaScript output targets the ECMAScript 2024 (ES15) standard, which is supported by every browser the platform aims at. ECMAScript modules are used throughout, which gives tree shaking on the production build and avoids the duplicated dependency problem that comes with CommonJS in Node.

The coding style follows the Next.js recommended ESLint configuration. The lint step runs on every commit and on every CI build, and the project does not deploy if linting fails. Component code follows the React conventions of named exports, functional components, and hooks for state management. Server actions are co located with the database access layer in the `actions/` directory, with each action exported as an async function that returns a typed `ActionResult` shape.

The database layer follows the SQL:2023 standard through PostgreSQL 16 on Neon. ACID guarantees are exploited by wrapping multi step writes inside transactions, particularly for the like and unlike actions where the leaderboard points must stay consistent with the like rows. Foreign keys and unique constraints are declared explicitly in the Drizzle schema and enforced by the database itself rather than only by the application code.

REST style ideas guide endpoint design where Server Actions are not the right fit, such as the Cloudinary signed upload endpoint and the Brevo webhook stub. Server actions themselves are not REST endpoints in the strict sense, since they are invoked by the framework directly without a path being declared explicitly, but they are organized following resource oriented thinking.

Two alternatives were ruled out. CommonJS modules were rejected in favour of ES modules because of tree shaking. Plain JavaScript was rejected in favour of TypeScript because the relational model has many cross referenced types and the compile time safety pays off across a project of this size.

5.1.2 Hardware Standards

The production deployment runs on a Hostinger virtual private server with two virtual CPUs, eight gigabytes of RAM, and a one hundred gigabyte NVMe SSD. The operating system is Ubuntu 22.04 LTS. This configuration is comfortable for the current expected user count and leaves plenty of headroom for growth. Node.js 20 LTS is the runtime, and PM2 is the process manager that keeps the application alive across crashes and reboots. Nginx sits in front of the Node process and terminates TLS using a Let's Encrypt certificate.

Development hardware was a personal computer with sixteen gigabytes of RAM and an Intel Core processor. Mobile testing was done on Android devices with screen sizes ranging from 5.5 inches to 6.7 inches and on the iOS simulator on a borrowed Mac. The combination has been sufficient to verify behaviour across the responsive breakpoints.

Vercel's serverless platform was the obvious alternative deployment. It would have removed all operating system level concerns and would have scaled automatically. The reason it was not chosen is cost. Vercel's free tier limits server action execution time and bandwidth in ways that interact badly with the Cloudinary signed upload step, and once the platform crosses the free tier the per execution pricing becomes harder to predict than a fixed monthly VPS bill. The Hostinger VPS gives full control over the runtime and has a known fixed cost.

5.1.3 Communication Standards

All client to server traffic travels over HTTPS using TLS 1.3, with the certificate issued by Let's Encrypt. HTTP/2 is enabled at the Nginx layer, which gives multiplexed connections and reduces latency for pages that load several small resources at once.

OAuth 2.0 with the Authorization Code flow handles the Google sign in. The PKCE extension is enabled by Better Auth so that the authorization code cannot be intercepted in transit. Session management itself uses secure HTTP only cookies that are scoped to the platform's domain.

Email goes through Brevo's REST API over HTTPS. The API key is kept in the server environment and is never shipped to the browser. Cloudinary uploads use signed presets that expire after a short window, so even if a preset is intercepted it cannot be reused.

The AI explanation calls go through the Vercel AI SDK to Google's Gemini API over HTTPS. The API key is again server side only. The image URLs sent to the model are public Cloudinary URLs, but they are not indexed publicly, and the model does not retain them.

WebSockets were considered for real time updates such as live like counts on a note that several users are looking at simultaneously. They were ruled out for now because TanStack Query's revalidation pattern produces a sufficient feel of

freshness without the operational complexity of a separate socket layer.

5.2 Impact on Society, Environment and Sustainability

5.2.1 Impact on Life

The most direct effect of Prepify is on the daily routines of university students preparing for exams. The current pattern of scrolling through years of WhatsApp messages, asking around in private chats, and accepting whatever quality of note happens to surface is replaced with a structured search through faculty, department, and course. A student who needs handwritten solutions for a specific course can find them in a few seconds, see who uploaded them, see the community's reaction to them, and read an AI generated structured explanation alongside the original photographs.

For the students who upload notes, the platform creates a lightweight reward loop. Their notes are credited to them by name, their leaderboard position rises with each like, their badges accumulate on their public profile, and their followers grow over time. None of these rewards involve money, but they produce real engagement during testing. Several test users uploaded a second note within an hour of seeing the first one approved.

For students who tend to remain quiet in big groups, Prepify removes the social cost of asking. A note request no longer needs to be typed into a public chat where everyone can see who is behind on which course. The student can quietly browse the platform, find what they need, and read the explanation without anyone else knowing.

5.2.2 Impact on Society and Environment

The platform encourages a culture of free knowledge sharing within university communities. The voting system surfaces the genuinely useful contributors rather than the loudest, and the leaderboard turns this into a small but visible recognition. Over time, this kind of structure tends to shift the social norm toward sharing rather than hoarding.

The environmental footprint is modest. The application runs on a single small VPS, the database is hosted on Neon's shared serverless infrastructure, and the images go through Cloudinary's CDN. None of these are heavy in absolute terms, and they all benefit from the underlying shared hosting model that uses energy more efficiently than dedicated hardware for a single user. The platform may also reduce paper use over time if students start storing and accessing notes digitally rather than photocopying handwritten pages or printing out solutions.

The platform is built around inclusivity by default. Anyone with a Google account can sign up. There is no fee, no credit currency, and no premium tier blocking access to the AI features. The dark mode option helps on OLED phone screens where it consumes less power than light mode. The responsive design works on cheap entry level Android phones, which are the dominant device in much of the country.

5.2.3 Ethical Aspects

Prepify collects only the personal data required to run the platform. From the Google OAuth response it stores the user's name, email, and profile picture URL. During onboarding it stores the user's chosen username, faculty, department, batch, and theme preference. It does not store passwords, since authentication goes through Google. It

does not sell user data to any third party. It does not run any analytics scripts that send personally identifiable information off platform.

Content uploaded to the platform passes through a moderation queue before becoming public. This protects the platform from spam, copyrighted textbook scans, and inappropriate material. Rejected uploads stay visible to the original uploader along with the rejection reason, which keeps the moderation process transparent.

The AI explanation feature does not retain user content beyond the model call. Image URLs are sent to Gemini at the moment the request is made, the structured response is written to the database, and no further training or retention happens on the model side. The provider documentation explicitly states that API content is not used for model training.

Users retain ownership of the content they upload. The terms of use grant the platform a non exclusive licence to display the content, which is what is needed to run a sharing platform, but the user can delete their notes at any time. Deleting a note cascades through the database and removes the associated files, resources, comments, likes, bookmarks, and AI explanations.

5.2.4 Sustainability Plan

The platform has been designed to be financially sustainable on a tight budget. The current monthly cost sits at roughly eight US dollars: about seven dollars for the Hostinger VPS, a fraction of a dollar for the AI explanation calls under current usage, and a small annual fee for the domain. Neon, Cloudinary, and Brevo all stay within their free tiers at the current load.

If the platform grows beyond the free tiers, several revenue paths become available. A non intrusive sponsorship strip on the home page could fund the AI calls. A donation link for active contributors would fit the spirit of the platform. An institutional licence for universities that want to use Prepify as their internal note sharing system could cover the hosting and operation costs. None of these require restricting the basic free experience for individual students.

The codebase has been written with future maintainers in mind. The TypeScript strict mode, the explicit Drizzle schemas, the named server actions, and the readable component file names should make it possible for a new developer to onboard in a few days. The provider abstraction in `lib/ai/provider.ts` makes it easy to switch AI vendors should the cost or quality balance shift in the future. The deployment runs on standard Linux tooling rather than on any vendor specific platform, which keeps the operational knowledge transferable.

5.3 Project Management and Financial Analysis

The project budget for Prepify is approximately 11,000 BDT. The breakdown is as follows: Neon PostgreSQL runs on the free tier with 0.5GB storage, of which approximately 100MB is currently used. Cloudinary operates on its free tier with 25GB storage, of which approximately 1GB is used for note images. The VPS hosting on Hostinger with 2 vCPU cores and 8GB RAM costs 10,000 BDT. The Brevo email service runs on the free tier supporting 300 emails per day, with current usage at approximately 10 emails per day. The domain name (`prepify.space`) costs 1,000 BDT for one year of registration. The total project cost comes to 11,000 BDT. An alternate budget using premium cloud services was considered: AWS EC2 t3.medium (approximately 3,500 BDT/month), AWS

RDS PostgreSQL (approximately 1,750 BDT/month), and AWS S3 with CloudFront (approximately 600 BDT/month), which would total roughly 70,000 BDT annually. The current budget was chosen because it provides adequate performance for the expected user base while keeping costs within a student developer's budget. The platform does not currently generate revenue, but potential revenue models include a freemium tier for unlimited AI processing credits, institutional licensing for universities, and non-intrusive advertising.

Table 5.1: Estimated Cost and Financial Analysis

Item	Usage Tier	Estimated Usage	Monthly Cost
Neon PostgreSQL	0.5GB storage (Free)	~100MB	Free
Clouinary	25GB storage (Free)	~1GB	Free
VPS Hosting	2vCPU cores, 8 GB Ram	1vCPU, 2GB Ram	10,000 BDT
Brevo Email	300 emails/day	~10 emails/day	Free
Domain Name	1 year registration	-	1000 BDT
			Total: 11,000 BDT

5.4 Complex Engineering Problem

This section maps the project to the complex engineering problem and engineering activities frameworks used by the FYDP committee. The mapping serves to verify that the work meets the depth, breadth, and innovation criteria expected of a final year design project.

5.4.1 Complex Problem Solving

Prepify qualifies as a complex engineering problem because it integrates multiple interconnected subsystems that must work reliably together. The authentication layer, the file storage layer, the relational database, the multimodal AI integration, the transactional points system, and the role based moderation layer each have their own consistency rules, and the platform only behaves correctly when all of them stay aligned.

Table 5.2: Mapping with Complex Engineering Problem.

EP1 Dept of Knowled ge	EP2 Range Of Conflicting Requireme nts	EP3 Depth of Analys is	EP4 Familiari ty of Issues	EP5 Extent of Applica ble Codes	EP6 Extent Of Stake- holder Involveme nt	EP7 Interdepende nce
✓		✓	✓	✓		✓

Justification of EP1 attainment: Required combined knowledge of full stack web development, relational database design with transactional integrity, OAuth 2.0 authentication flows, multimodal large language model integration, prompt engineering, and CDN based asset delivery.

Justification of EP3 attainment: The system design considered multiple alternatives at every architectural level (frontend framework, database, ORM, authentication, image hosting, AI provider) and chose each one with documented trade offs around cost, fit, and maintainability.

Justification of EP4 attainment: Several aspects of the project crossed into less familiar territory, including multimodal model integration, structured output enforcement through Zod, and the regeneration cap design. These had to be learned during the project rather than relying on prior knowledge.

Justification of EP5 attainment: The work follows international standards including ECMAScript 2024, TypeScript strict mode, OAuth 2.0, TLS 1.3, HTTP/2, SQL:2023, and the IEEE referencing format used in this report.

Justification of EP7 attainment: The subsystems are tightly interdependent. A like updates the leaderboard which feeds the badges check which updates the profile. A note approval triggers email and notification. An AI explanation depends on the moderation status. These interdependencies are explicit in the database schema and the server actions.

Mapping with Knowledge Profile

Table 5.3: Mapping with knowledge Profile.

K1 Natu ral Scien ce	K2 Mathem atics	K3 Engineeri ng Fundame ntals	K4 Special ist Knowle dge	K5 Enginee ring Design	K6 Enginee ring Practice	K7 Comprehe nsion	K8 Resear ch Literat ure
		✓	✓	✓	✓		

Justification of Knowledge Profile (K3, K4, K5, K6, K8) for EP1

The depth of knowledge required for the project maps clearly to several elements of the engineering knowledge profile.

K3 (Engineering Fundamentals) was applied through the relational database design with explicit foreign key constraints and unique constraints, the use of transactions to keep multi step writes consistent, and the application of REST and Server Actions design patterns to the network layer.

K4 (Specialist Knowledge) was demonstrated through the integration of the multimodal large language model (Gemini 2.5 Flash) for direct image analysis, the use of the Vercel AI SDK's structured output enforcement, and the prompt design for handwritten academic content.

K5 (Engineering Design) shows up in the modular system architecture that separates the social platform layer, the AI pipeline, and the gamification layer so that any one of them can evolve without restructuring the others. The provider abstraction in lib/ai/provider.ts is a small but pointed example of design for change.

K6 (Engineering Practice) is reflected throughout the work in the use of industry standard tooling: Next.js, PostgreSQL, OAuth 2.0, Cloudinary. The deployment process uses standard Linux operations rather than vendor specific platforms, which keeps the practice transferable.

5.4.2 Engineering Activities

The project also maps to several of the complex engineering activities defined in the framework.

Mapping with Complex Engineering Activities

EA1 (Range of Resources) applies because the project uses diverse resources including cloud infrastructure, third-party APIs, open-source libraries, and AI services that must be coordinated effectively. EA2 (Level of Interaction) is relevant because the platform integrates multiple technical systems: the Next.js application server, PostgreSQL database, Cloudinary CDN, Google OAuth, Brevo email API, and AI model APIs, each with different interfaces and protocols. EA3 (Innovation) is demonstrated through the novel combination of handwritten note sharing with community ranking and AI-powered content enhancement, which no existing platform offers. EA4 (Consequences for Society and Environment) is addressed through the platform's positive impact on educational equity and the ethical handling of user data as discussed in the impact analysis. EA5 (Familiarity) indicates a moderately familiar problem domain (note sharing) combined with less familiar technical challenges (OCR on handwriting, LLM integration for content

restructuring).

Table 5.4: Mapping with Complex Engineering Activities.

EA1 Range of re- sources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
✓	✓	✓	✓	✓

Justification of EA1 attainment: The project uses cloud database (Neon), CDN (Clouinary), email API (Brevo), authentication provider (Google OAuth), AI service (Gemini), VPS (Hostinger), open source frameworks (Next.js, React, Drizzle), and version control (GitHub). All of them have to be coordinated.

Justification of EA2 attainment: The platform integrates multiple services across different protocols. The Next.js application talks to Postgres, Clouinary, Brevo, Google OAuth, and the Gemini API simultaneously, each with different shapes and reliability characteristics.

Justification of EA3 attainment: The combination of free, community ranked, handwriting focused note sharing with multimodal AI explanation is novel in the local context and is not currently offered by any commercial platform. The decision to skip a separate OCR step in favour of direct multimodal inference is also a non obvious design choice.

Justification of EA4 attainment: The platform improves educational equity by making quality study materials accessible to all students regardless of social network. It also handles user data ethically and runs on a low energy infrastructure footprint.

Justification of EP5 attainment: Note sharing as a problem domain is familiar, but the technical territory of multimodal LLM integration with structured output enforcement is new and required learning during the project.

5.5 Summary

This chapter walked through the engineering standards that the project follows, the impact on its users and on the wider environment, the financial picture, and the mapping to complex engineering problem and activities frameworks. The platform follows established standards across software, hardware, and communication layers. It improves educational equity for its target users while keeping its environmental and financial footprint modest. It maps cleanly to the depth of knowledge, range of resources, and innovation criteria expected of a final year design project. The next chapter closes the report with a summary of what has been delivered, the limitations of the current implementation, and the directions for future work.

Chapter 6

Conclusion

This chapter summarizes the work completed, discusses the limitations of the current implementation, and outlines directions for future development.

6.1 Summary

Prepify has been built as a final year design project to address the disorganized state of handwritten note sharing among university students in Bangladesh. The work has produced a complete, deployed web platform that combines a community driven note sharing layer, a moderation system, a points based contributor leaderboard, study streaks and achievement badges, bookmarks and curated collections, and an AI explanation pipeline that reads handwritten note images directly through a multimodal large language model.

The platform is built on Next.js 16.1 with React 19.2 on the frontend, Drizzle ORM over Neon serverless PostgreSQL on the backend, Cloudinary for image hosting and delivery, Better Auth with Google OAuth for authentication, the Brevo API for transactional email, and Google's Gemini 2.5 Flash through the Vercel AI SDK for the AI explanations. The full source is in TypeScript with strict mode enabled, runs through ESLint cleanly, and is deployed to a Hostinger VPS using PM2 and Nginx with Let's Encrypt TLS.

Phase 1 of the development covered authentication, multi step onboarding, multi image note upload with Cloudinary signed presets, the like and comment systems with optimistic updates, the leaderboard with transactional points, follows with notifications and email, the moderation queue with role based dashboards, and the moderator application flow. Phase 2 added bookmarks and collections (private or public), the streaks system that tracks daily activity, the badges system across upload, streak, engagement, and social categories, and the AI explanation pipeline that returns a structured response containing summary, steps, key concepts, observations, and topic tags.

All of the project objectives defined in Chapter 1 have been met. The platform is free at the point of use, focused on photographed handwritten content, organized around the Bangladeshi university taxonomy, AI assisted through multimodal explanations, and responsive across screen sizes with a system aware dark mode. All testing was performed by hand rather than through an automated test suite, and these manual walkthroughs across several test accounts confirmed that every major user flow behaves as designed and that the regeneration cap on AI explanations works correctly. The production deployment has scored above 85 on Lighthouse for performance and above 90 for accessibility, and the platform's total monthly operating cost remains around eight US

dollars.

6.2 Limitation

While the platform is functional in every promised area, several limitations remain that should be acknowledged honestly.

The first limitation is the absence of an automated test suite. There are no unit tests around the server actions, no integration tests around the database transactions, and no Playwright or Cypress scripts for the user interface. Every feature has been validated through manual walkthroughs in the browser and by inspecting the database directly through Drizzle Studio, which is reliable in the short term but does not scale. As the codebase grows, a regression in a far away module could go unnoticed until a manual sweep happens to catch it. Adding a small base of tests around the most critical flows (auth, note upload, leaderboard transactions, AI regeneration cap) would give a much faster safety net for future changes.

The second limitation is the size of the active user base. The platform has been exercised by a small group of test users rather than by hundreds of real students. Although every major flow has been verified manually, behaviour under high concurrency or under aggressive write loads has not been observed in practice. Stress testing with simulated traffic at the level of a few hundred concurrent users would be a useful next step, and would also help justify investing in the automated test suite mentioned above.

The third limitation is the AI explanation quality on very poor handwriting. Gemini 2.5 Flash handles clear handwriting well, including mixed Bangla and English content, but on very faint pencil writing or on heavily corrected pages it can produce incomplete summaries. The model does flag the parts it cannot read, which is the correct behaviour, but a fallback that prompts the user to re upload a clearer photograph would be a sensible improvement.

The fourth limitation concerns the moderation throughput. Every uploaded note currently waits in the queue until a moderator reviews it. If the platform grows faster than the moderation team, this queue could become a bottleneck. A first pass automated check using a smaller model to flag obviously off topic uploads could relieve some of this pressure.

The fifth limitation is that the platform currently supports image and PDF based notes only. Markdown notes, and Word documents are not yet accepted. There are valid reasons to keep the focus on handwriting.

The sixth limitation is the absence of in app push notifications. Notifications are delivered through the in app bell and through email, but there is no Web Push integration that would let students receive a notification when the platform is closed.

The seventh limitation is the leaderboard scoring algorithm. It currently treats every like as one point regardless of when it was given or who gave it. A more sophisticated algorithm could weight recent likes more heavily, give more weight to likes from established contributors, or decay the points of older notes so that new uploaders are not permanently behind the early adopters.

The eighth limitation is the lack of a mobile application. The responsive web interface works well on phones, but a native or React Native application would provide better camera integration for capturing notes directly into the upload form.

6.3 Future Work

Several directions for future work emerge from the limitations above and from the broader vision for the platform.

The most immediate priority is to expand the AI feature set. The current explanation feature is a strong base. A natural next step is automated quiz generation, where the platform produces a small set of practice questions from a note's content and lets the student answer them inside the platform. The Gemini API supports this naturally, and the regeneration cap can be reused to control cost. A second AI feature worth exploring is similar note discovery, where the platform uses embeddings of the AI generated topic tags and summaries to recommend related notes from the same course or from related courses. PDF and document support is a clear next step. The Vercel AI SDK already supports document inputs, and Cloudinary handles PDF storage natively. Adding a parallel upload path for PDFs would broaden the platform's coverage of academic content without restructuring the existing image flow.

Web Push notifications would close the gap between in app notifications and email. The browser's Web Push API allows a service worker to receive a notification even when the platform tab is closed, which would significantly improve engagement during exam season when students spend a lot of time switching between apps.

A native mobile companion is a longer term goal. A React Native build would share most of the type definitions and validation logic with the web codebase and would provide a much better camera experience for capturing handwritten notes directly. A native build also opens the door to offline reading of bookmarked notes.

Building out an automated test suite is a clear next step on the engineering side. A small Vitest base around the most critical server actions (auth, note upload, leaderboard transactions, AI regeneration cap), an integration layer that runs against a disposable Postgres instance to verify the database transactions, and a Playwright suite that walks through the half dozen most important user flows in a real browser would together replace the bulk of the current manual test sweep. With those in place, every push to the main branch could be guarded by a continuous integration pipeline so that regressions are caught before they reach production.

Smarter ranking and recommendation is another area worth investing in. The current leaderboard and feed sort are simple. A more nuanced system that combines recency, similarity to the user's department, and the user's reading history would produce a feed that feels more personal. The same embedding approach used for similar note discovery could feed into this.

Multi institution support is the largest single direction for future growth. The current platform is seeded with Daffodil International University as the example institution. Extending the same model to other Bangladeshi universities, each with their own faculty, department, and course taxonomy, would multiply the platform's reach without changing its core mechanics. Institutional verification through a domain restricted email check could keep each institution's content reliably tagged.

Collaborative annotation on shared notes is a stretch goal. A model where multiple students could highlight passages on a shared note and leave inline comments would turn

Prepify from a sharing platform into a study group platform. This would require significant changes to the note detail page and to the database schema, but it would unlock a use case that no comparable platform currently supports.

The final item on the future work list is open sourcing the platform. Releasing the codebase under a permissive licence, with proper documentation and a contributor guide, would invite other students and university IT teams to extend the platform for their own institutions. Combined with institutional licences for universities that prefer a managed deployment, this would set the platform on a path to long term sustainability beyond a single student's free year project.

References

- [1] [1] M. A. Chatti, M. R. Agustawan, M. Jarke, and M. Specht, "Toward a Personal Learning Environment Framework," *International Journal of Virtual and Personal Learning Environments*, vol. 1, no. 4, pp. 66-85, 2010. DOI: 10.4018/jvple.2010100105.
- [2] [2] G. Attwell, "Personal Learning Environments - the future of eLearning?," *eLearning Papers*, vol. 2, no. 1, pp. 1-8, 2007. ISSN: 1887-1542.
- [3] [3] S. Getenet, R. Cantle, P. Redmond, and P. Albion, "Students digital technology attitude, literacy and self-efficacy and their effect on online learning engagement," *International Journal of Educational Technology in Higher Education*, vol. 21, no. 1, Article 3, 2024. DOI: 10.1186/s41239-023-00437-y.
- [4] [4] Ocenot, "Buy and Sell Academic Notes Online," Ocenot.com. Available: <https://ocenot.com/>. [Accessed: 20-Jan-2026].
- [5] [5] Docsity, "Study Notes and University Resources," Docsity.com. Available: <https://www.docsity.com/>. [Accessed: 20-Jan-2026].
- [6] [6] Thinkswap, "Study Notes and Learning Resources," Thinkswap.com. Available: <https://www.thinkswap.com/>. [Accessed: 20-Jan-2026].
- [7] [7] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," arXiv preprint arXiv:2312.11805, 2023.
- [8] [8] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Doc. Anal. Recognit. (ICDAR)*, Curitiba, Brazil, 2007, vol. 2, pp. 629-633.
- [9] [9] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [10][10] Vercel, "Next.js Documentation," 2025. [Online]. Available: <https://nextjs.org/docs>.